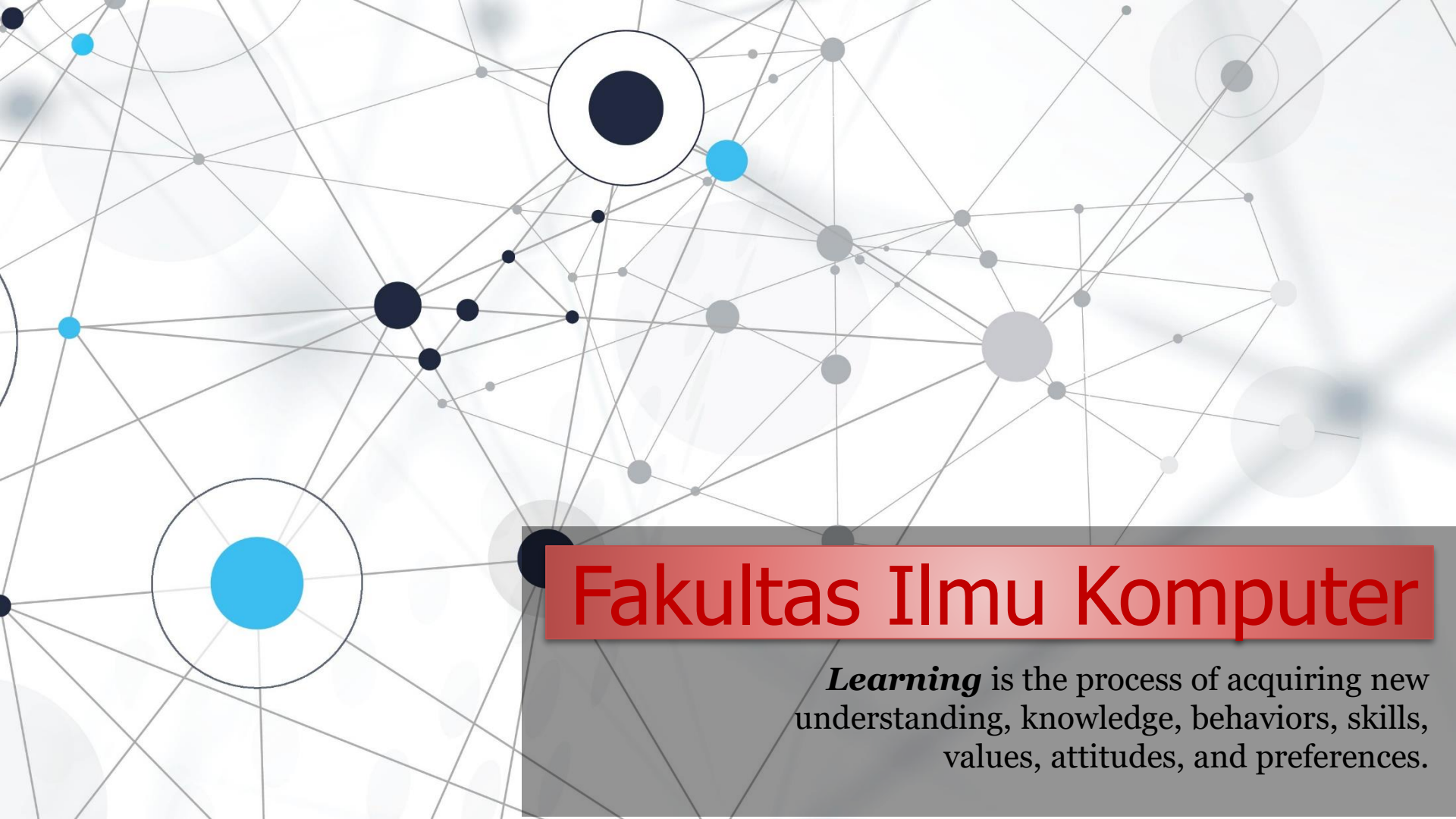


An abstract network diagram with various sized nodes (black, blue, grey) connected by thin grey lines. Some nodes are highlighted with larger circles. The background is white with faint grey circles.

# Fakultas Ilmu Komputer

***Learning*** is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

## Special Case

***[Python Tkinter & Flask Frameworks]***



# Framework Programming



- ❑ A ***programming framework*** is a prepackaged set of solutions that solves common development problems.
- ❑ *Why do people use frameworks?* I can say that the **No. 1** reason for using a framework is probably *saving time*. But as you'll soon see, there are several other benefits you can get by leveraging frameworks.
- ❑ Web ***Application Framework*** or simply Web Framework represents a collection of libraries and modules that enables a web application developer to write applications without having to bother about low-level details such as protocols, thread management etc.



What do you *think* about  
Framework Programming?

# Framework MVC

| Bahasa     | Framework                          |
|------------|------------------------------------|
| PHP        | CodeIgniter, Laravel, Yii, dll     |
| Java       | Spring MVC, JSF, Struts, dll       |
| Python     | Django, CherryPy, dll              |
| Ruby       | Ruby on Rails, Sinatra ,dll        |
| Javascript | AngularJS, React, Backbone.js, dll |

A decorative border of green leaves and branches at the top of the page.

# 1 - Install **Tkinter** *on* Windows [Graphical User Interfaces (GUIs)]

A decorative border of green leaves and branches at the bottom of the page.



# Tkinter Installation on Windows



---

Tkinter is a ***package*** in Python to create Graphical User interfaces (GUIs). Installing Tkinter on Windows is as simple as installing Python 3.x since Tkinter is included in the **Python 3 core**.

# What is Tkinter in Python?

- ❑ Python has a lot of **GUI Frameworks**, but Tkinter is the only framework that's built into the *Python standard library*.
- ❑ Tkinter has several strengths. It's cross-platform, so the same code works on Windows, macOS, and Linux.
- ❑ Visual elements are rendered using native operating system elements, so applications built with Tkinter look like they belong on the platform where they're run.



# Take Advantage of Latest Version of Tkinter

- ❑ Need to install a version of Python that supports Tcl/Tk 8.5 or greater, as well as Ttk (*Tile extension integrated into Tk*), which is required in order to run the current Tk widget set.
- ❑ Python 3.7 or greater for Windows comes full packages to run Tk fully configured out-of-the-box.
- ❑ If you have **Python 3.6** or an earlier version, you can simply upgrade to a newer Python version that includes the latest version of Tkinter.

# Follow these steps

1) Install Anaconda Python, Jupyter Notebook And Spyder on Windows 10  
<https://www.youtube.com/watch?v=g6ln1dAt-RI>

2) Conda create new python environment  
<https://www.youtube.com/watch?v=k-MWeAWEta8>  
<https://www.activestate.com/resources/quick-reads/how-to-install-tkinter-in-windows/>

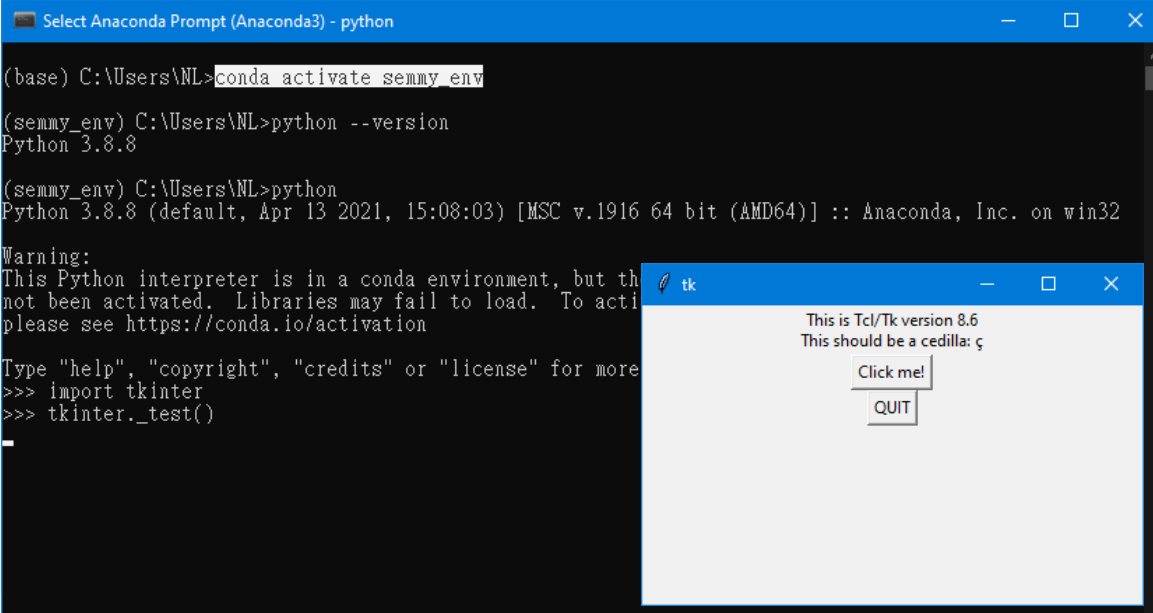
3) Activate the new environment that has Python 3.8  
*\$ conda activate <env\_name>*

4) Check Python version  
*\$ python --version*

5) Install Tkinter on windows  
*\$ pip install tk*

6) Verify tkinter installation

```
>>> import tkinter
>>> tkinter._test()
```

The screenshot shows two overlapping windows. The background window is the 'Select Anaconda Prompt (Anaconda3) - python' terminal. It displays the command 'conda activate semmy env' being executed, followed by 'python --version' which returns 'Python 3.8.8'. Then, 'python' is entered, showing the Python 3.8.8 shell prompt. A warning message is shown: 'Warning: This Python interpreter is in a conda environment, but the environment has not been activated. Libraries may fail to load. To activate this environment, please see https://conda.io/activation'. Below the warning, the user enters 'import tkinter' and 'tkinter.\_test()'. The foreground window is a Tkinter window titled 'tk'. It contains the text 'This is Tcl/Tk version 8.6' and 'This should be a cedilla: ç', followed by two buttons: 'Click me!' and 'QUIT'.

# Working With Tkinter Widgets

| Widget Class | Description                                                                           | Some of the widgets available |
|--------------|---------------------------------------------------------------------------------------|-------------------------------|
| Label        | A widget used to display text on the screen                                           |                               |
| Button       | A button that can contain text and can perform an action when clicked                 |                               |
| Entry        | A text entry widget that allows only a single line of text                            |                               |
| Text         | A text entry widget that allows multiline text entry                                  |                               |
| Frame        | A rectangular region used to group related widgets or provide padding between widgets |                               |

Widgets are the bread and butter of the Python GUI framework Tkinter. They are the elements through which users interact with your program. Each widget in Tkinter is defined by a class.

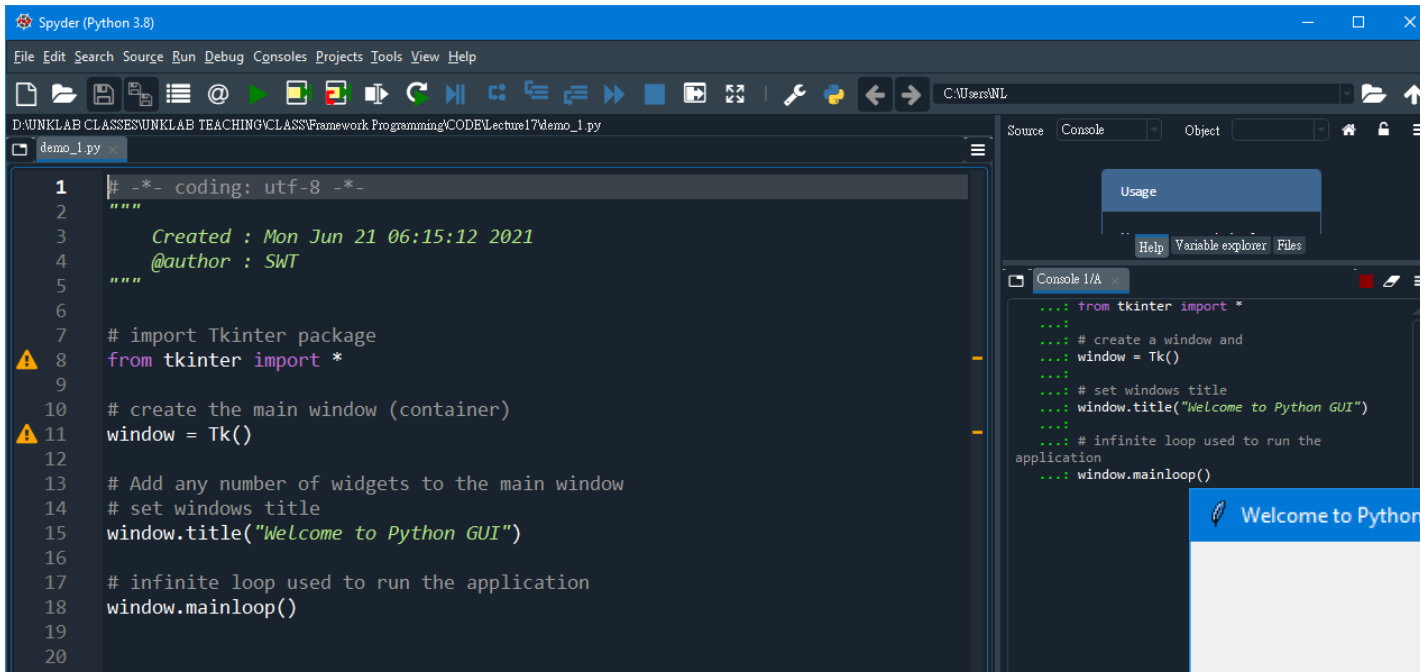


(#1) exercise for students  
**[GUI Framework]**

# Exercise #1

File : *demo\_1.py*

Desc : Display/create first GUI application with python

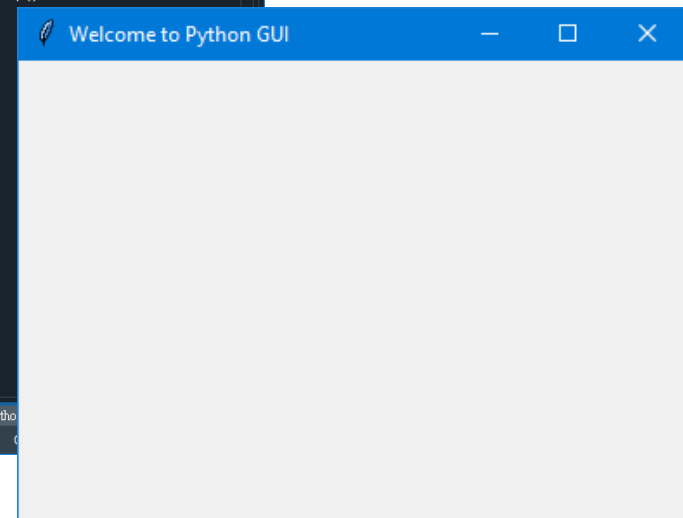


```
1  -*- coding: utf-8 -*-
2  """
3      Created : Mon Jun 21 06:15:12 2021
4      @author : SWT
5  """
6
7  # import Tkinter package
8  from tkinter import *
9
10 # create the main window (container)
11 window = Tk()
12
13 # Add any number of widgets to the main window
14 # set windows title
15 window.title("Welcome to Python GUI")
16
17 # infinite loop used to run the application
18 window.mainloop()
19
20
```

Console I/A

```
...: from tkinter import *
...:
...: # create a window and
...: window = Tk()
...:
...: # set windows title
...: window.title("Welcome to Python GUI")
...:
...: # infinite loop used to run the
...: application
...: window.mainloop()
```

Output Program:



Last line which calls the **mainloop function**, this function calls the endless loop of the window, so the window will wait for any user interaction till we close it. If you forget to call the mainloop function, nothing will appear to the user.



# Remember: “*two main methods*”

Users need to remember while creating the Python application with GUI.

1. **Tk(screenName=None, baseName=None, className='Tk', useTk=1)**: To create a main window, tkinter offers a method 'Tk(screenName=None, baseName=None, className='Tk', useTk=1)'. To change the name of the window, you can change the className to the desired one. The basic code used to create the main window of the application is:

```
m=tkinter.Tk() where m is the name of the main window object
```

2. **mainloop()**: There is a method known by the name mainloop() is used when your application is ready to run. mainloop() is an infinite loop used to run the application, wait for an event to occur and process the event as long as the window is not closed.

```
m.mainloop()
```



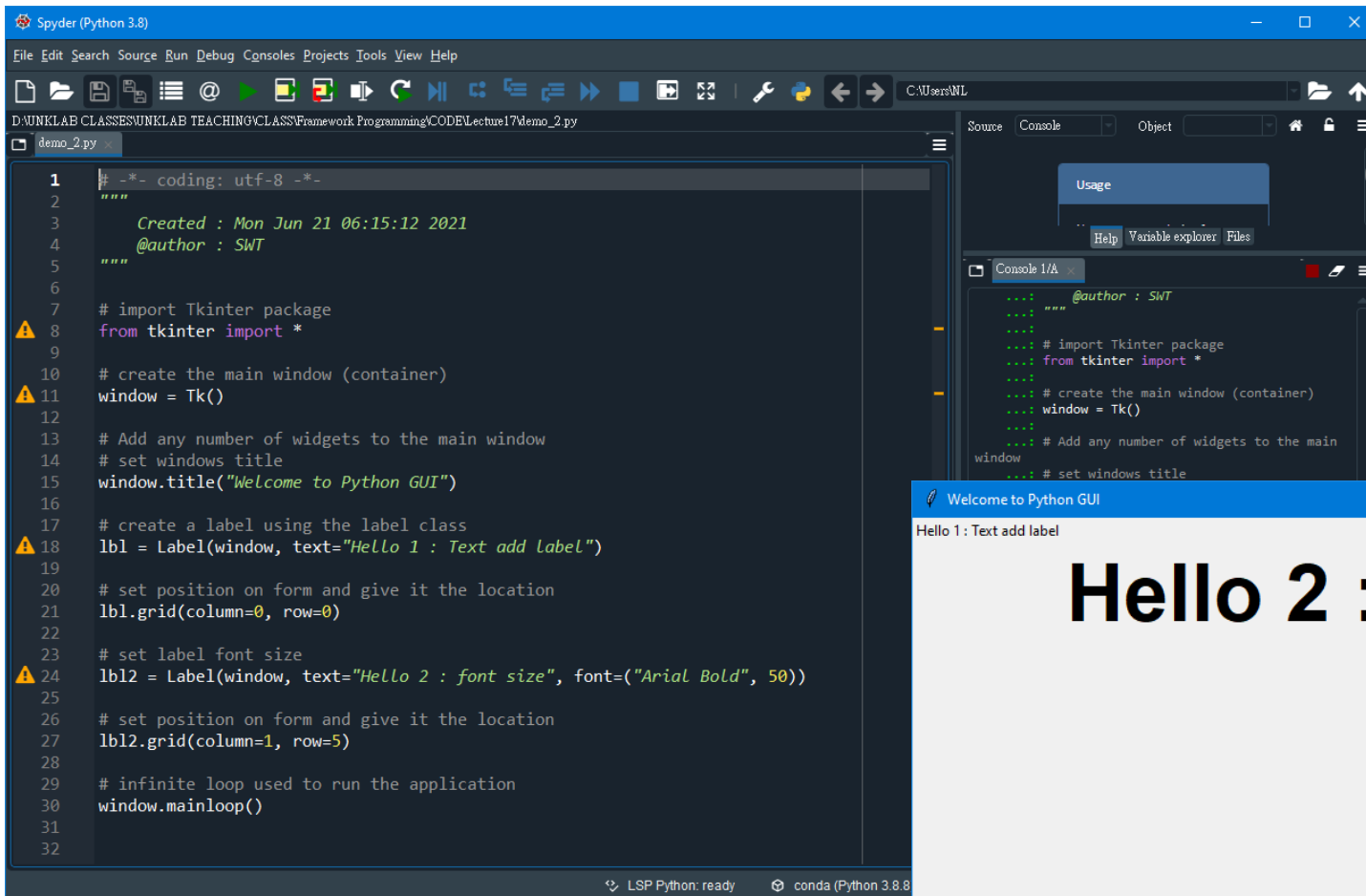
```
import tkinter
m = tkinter.Tk()
...
widgets are added here
...
m.mainloop()
```

Importing tkinter is same as importing any other module in the Python code. Note that the name of the module in Python 2.x is '**Tkinter**' and in Python 3.x it is '**tkinter**'.

# Exercise #2

File : *demo\_2.py*

Desc : Create a label widget



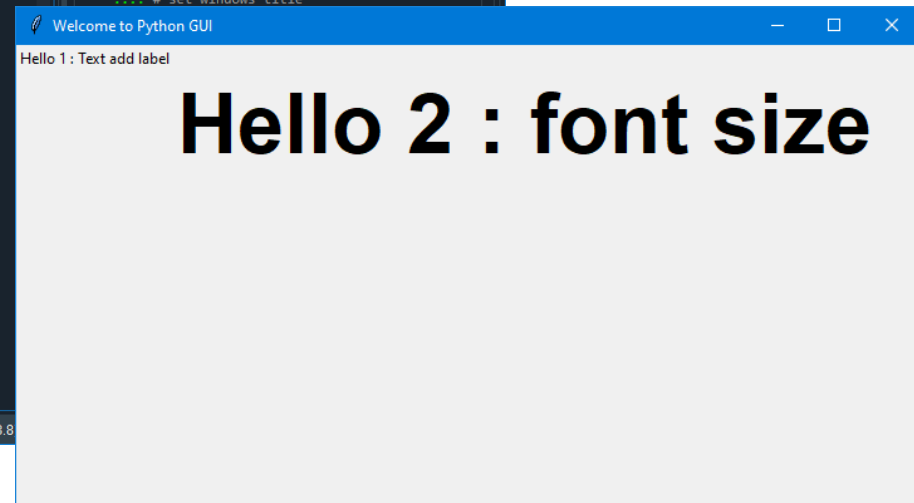
The screenshot shows the Spyder Python IDE interface. The left pane displays the code for `demo_2.py`, which includes comments about the file's creation and author, imports the Tkinter package, creates a main window titled "Welcome to Python GUI", and adds two labels. The right pane shows the console output, which repeats the code comments. The bottom status bar indicates "LSP Python: ready" and "conda (Python 3.8.8)".

```
1  -*- coding: utf-8 -*-
2  """
3      Created : Mon Jun 21 06:15:12 2021
4      @author : SWT
5  """
6
7  # import Tkinter package
8  from tkinter import *
9
10 # create the main window (container)
11 window = Tk()
12
13 # Add any number of widgets to the main window
14 # set windows title
15 window.title("Welcome to Python GUI")
16
17 # create a label using the label class
18 lbl1 = Label(window, text="Hello 1 : Text add Label")
19
20 # set position on form and give it the location
21 lbl1.grid(column=0, row=0)
22
23 # set label font size
24 lbl2 = Label(window, text="Hello 2 : font size", font=("Arial Bold", 50))
25
26 # set position on form and give it the location
27 lbl2.grid(column=1, row=5)
28
29 # infinite loop used to run the application
30 window.mainloop()
31
32
```

Console 1/A

```
...: @author : SWT
...: """
...: # import Tkinter package
...: from tkinter import *
...: # create the main window (container)
...: window = Tk()
...: # Add any number of widgets to the main
window
...: # set windows title
```

Output Program:

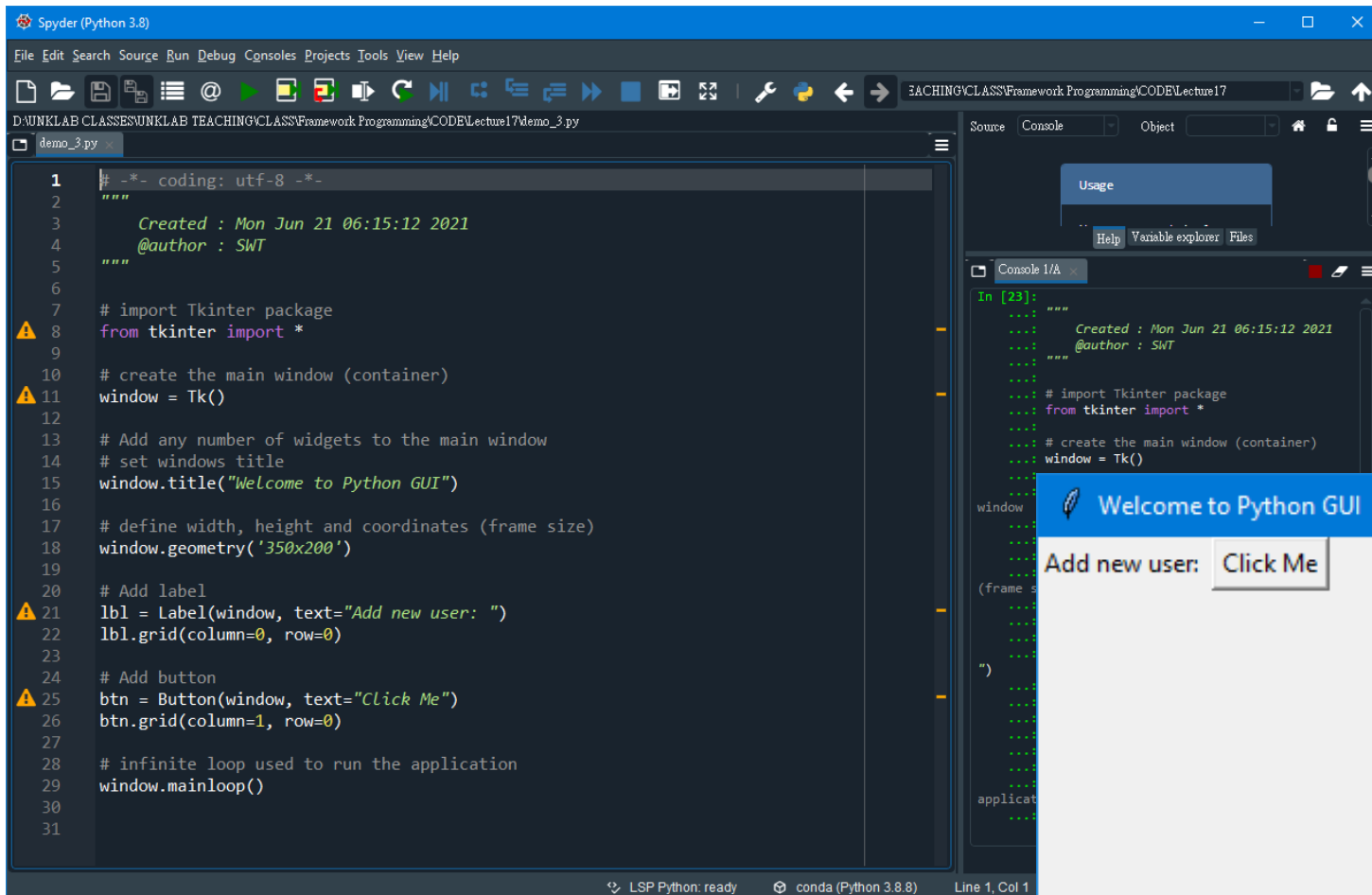




# Exercise #3

File : *demo\_3.py*

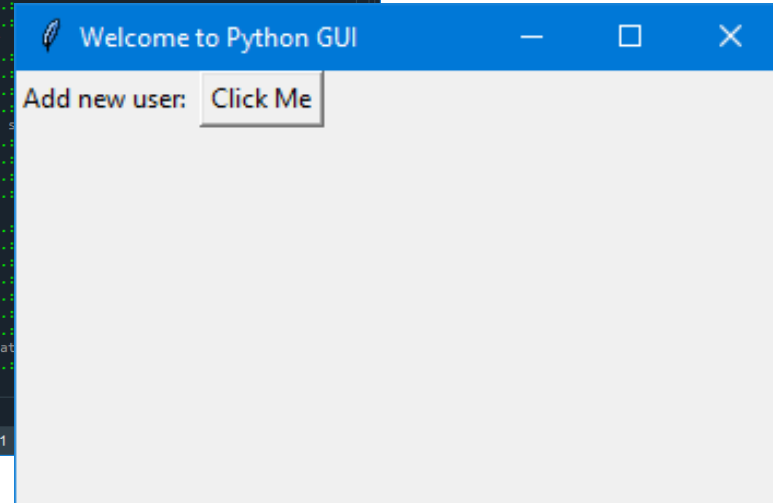
Desc : Create a button and add a button widget



The screenshot shows the Spyder Python IDE interface. The left pane displays the code for `demo_3.py`, which creates a Tkinter window titled "Welcome to Python GUI" with a label "Add new user:" and a button "Click Me". The right pane shows the console output, which matches the code. The bottom status bar indicates "LSP Python: ready", "conda (Python 3.8.8)", and "Line 1, Col 1".

```
1  -*- coding: utf-8 -*-
2  """
3      Created : Mon Jun 21 06:15:12 2021
4      @author : SWT
5  """
6
7  # import Tkinter package
8  from tkinter import *
9
10 # create the main window (container)
11 window = Tk()
12
13 # Add any number of widgets to the main window
14 # set windows title
15 window.title("Welcome to Python GUI")
16
17 # define width, height and coordinates (frame size)
18 window.geometry('350x200')
19
20 # Add label
21 lbl = Label(window, text="Add new user:")
22 lbl.grid(column=0, row=0)
23
24 # Add button
25 btn = Button(window, text="Click Me")
26 btn.grid(column=1, row=0)
27
28 # infinite loop used to run the application
29 window.mainloop()
30
31
```

**Output Program:**



# Exercise #4

File : *demo\_4.py*

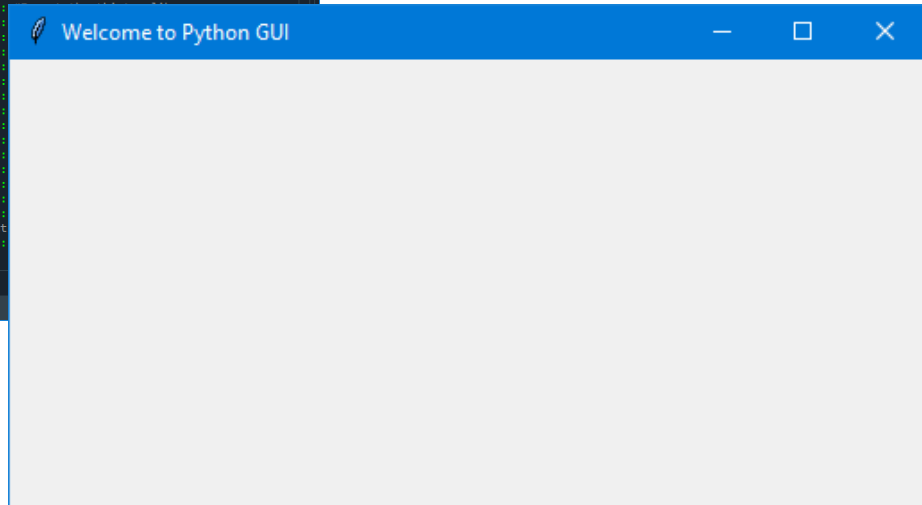
### Desc : How to center a window on the screen in Tkinter?

The screenshot displays the Spyder Python IDE interface. The top menu bar includes File, Edit, Search, Source, Run, Debug, Consoles, Projects, Tools, View, and Help. Below the menu is a toolbar with icons for file operations, running code, and debugging. The main editor window shows a Python script named 'demo\_4.py' with the following content:

```
1 # -*- coding: utf-8 -*-  
2 """  
3     Created : Mon Jun 21 06:15:12 2021  
4     @author : SWT  
5 """  
6  
7 #Import the tkinter library  
8 from tkinter import *  
9  
10 #Create an instance of tkinter frame  
11 window = Tk()  
12  
13 # set windows title  
14 window.title("Welcome to Python GUI")  
15  
16 #Set the geometry  
17 window.geometry("600x250")  
18  
19 window.eval('tk::PlaceWindow . center')  
20  
21 # infinite loop used to run the application  
22 window.mainloop()  
23  
24
```

The right sidebar contains three panels: 'Usage' (empty), 'Help' (showing 'Variable explorer' and 'Files'), and 'Console 1/A'. The console output shows the execution of the script, including the creation of the window and the title 'Welcome to Python GUI'. At the bottom status bar, it indicates 'LSP Python: ready', 'conda (Python 3.8.8)', and 'Line 1, Col 1'.

### Output Program:

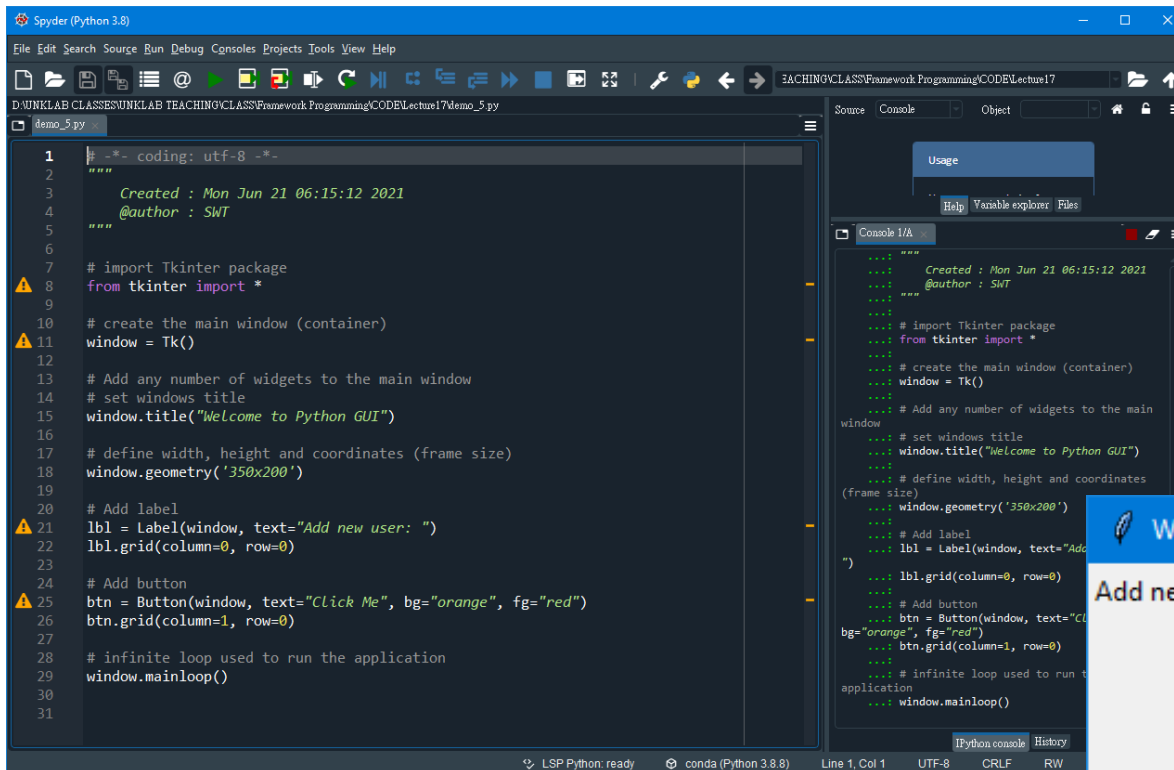


In order to place a tkinter window at the center of the screen, we can use the `PlaceWindow` method in which we can pass the toplevel window as an argument and add it into the center.

# Exercise #5

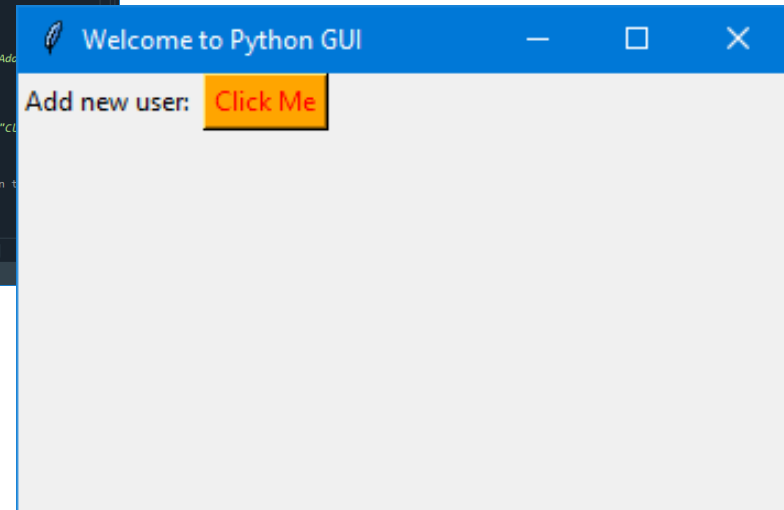
File : *demo\_5.py*

Desc : Change button foreground and background colors



```
1  -*- coding: utf-8 -*-
2  """
3      Created : Mon Jun 21 06:15:12 2021
4      @author : SWT
5  """
6
7  # import Tkinter package
8  from tkinter import *
9
10 # create the main window (container)
11 window = Tk()
12
13 # Add any number of widgets to the main window
14 # set windows title
15 window.title("Welcome to Python GUI")
16
17 # define width, height and coordinates (frame size)
18 window.geometry('350x200')
19
20 # Add label
21 lbl = Label(window, text="Add new user: ")
22 lbl.grid(column=0, row=0)
23
24 # Add button
25 btn = Button(window, text="Click Me", bg="orange", fg="red")
26 btn.grid(column=1, row=0)
27
28 # infinite loop used to run the application
29 window.mainloop()
30
31
```

Output Program:



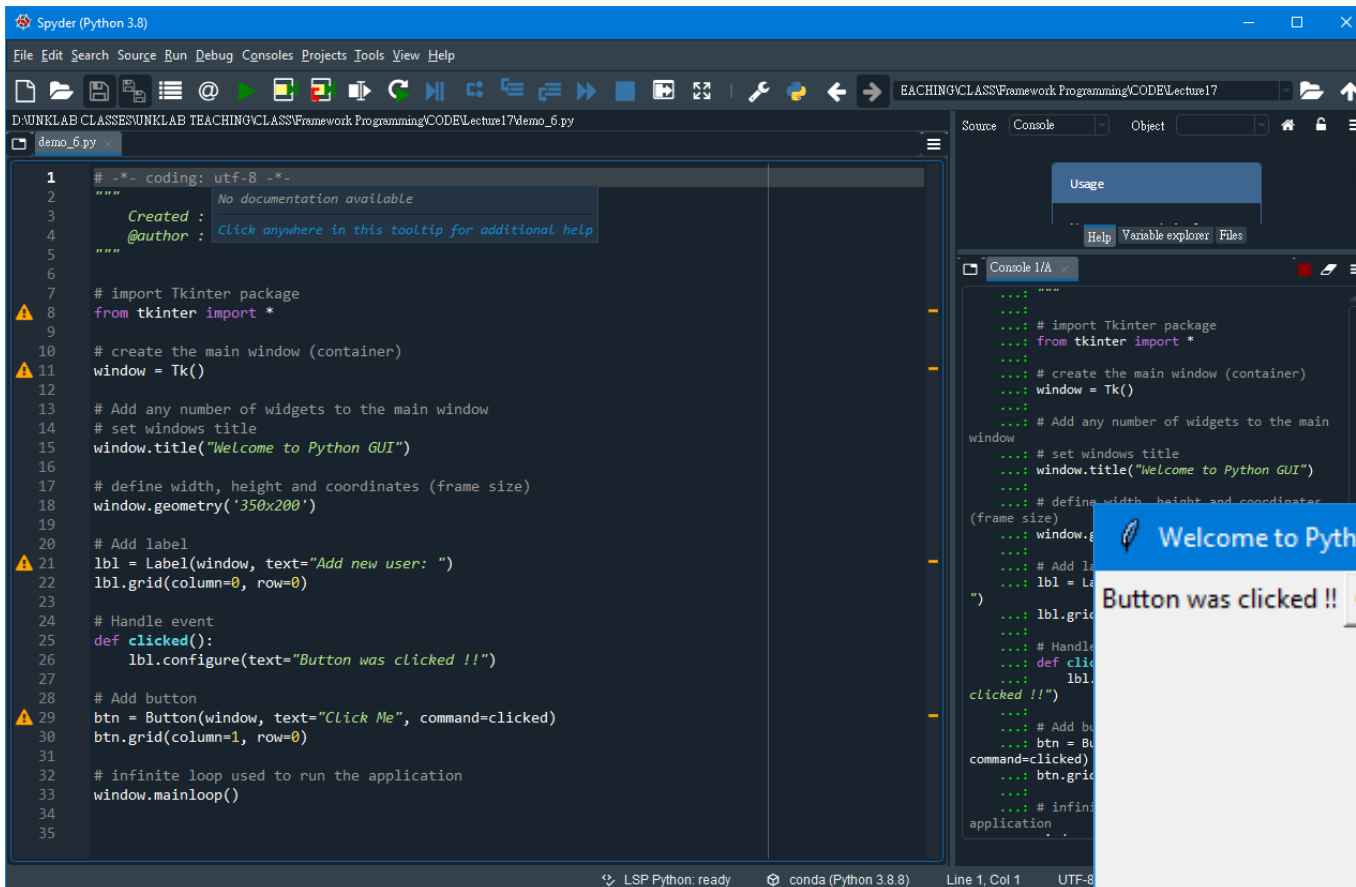
Set the following important properties to customize a button:

- 1) **text** : caption of the button
- 2) **bg** : background colour
- 3) **fg** : foreground colour
- 4) **font** : font name and size
- 5) **image** : to be displayed instead of text
- 6) **command** : function to be called when clicked

# Exercise #6

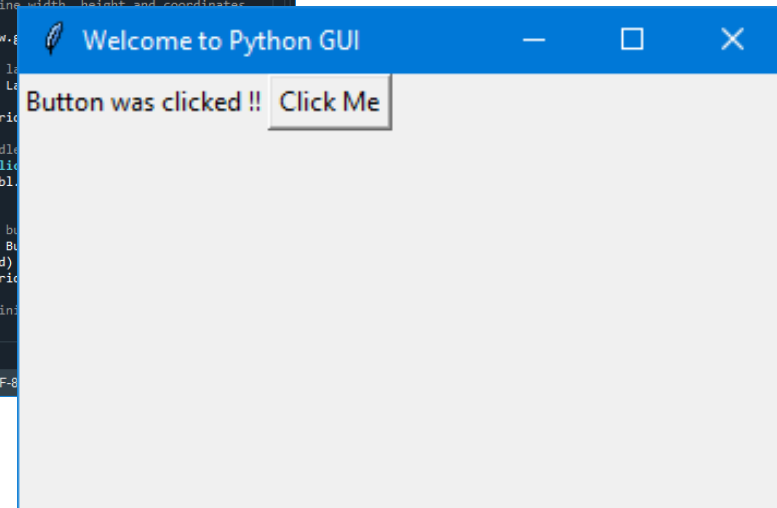
File : *demo\_6.py*

Desc : Handle button click event



```
1  -*- coding: utf-8 -*-
2  """
3  Created :
4  @author : Click anywhere in this tooltip for additional help
5  """
6
7  # import Tkinter package
8  from tkinter import *
9
10 # create the main window (container)
11 window = Tk()
12
13 # Add any number of widgets to the main window
14 # set windows title
15 window.title("Welcome to Python GUI")
16
17 # define width, height and coordinates (frame size)
18 window.geometry('350x200')
19
20 # Add label
21 lbl = Label(window, text="Add new user: ")
22 lbl.grid(column=0, row=0)
23
24 # Handle event
25 def clicked():
26     lbl.configure(text="Button was clicked !!")
27
28 # Add button
29 btn = Button(window, text="Click Me", command=clicked)
30 btn.grid(column=1, row=0)
31
32 # infinite loop used to run the application
33 window.mainloop()
34
35
```

Output Program:

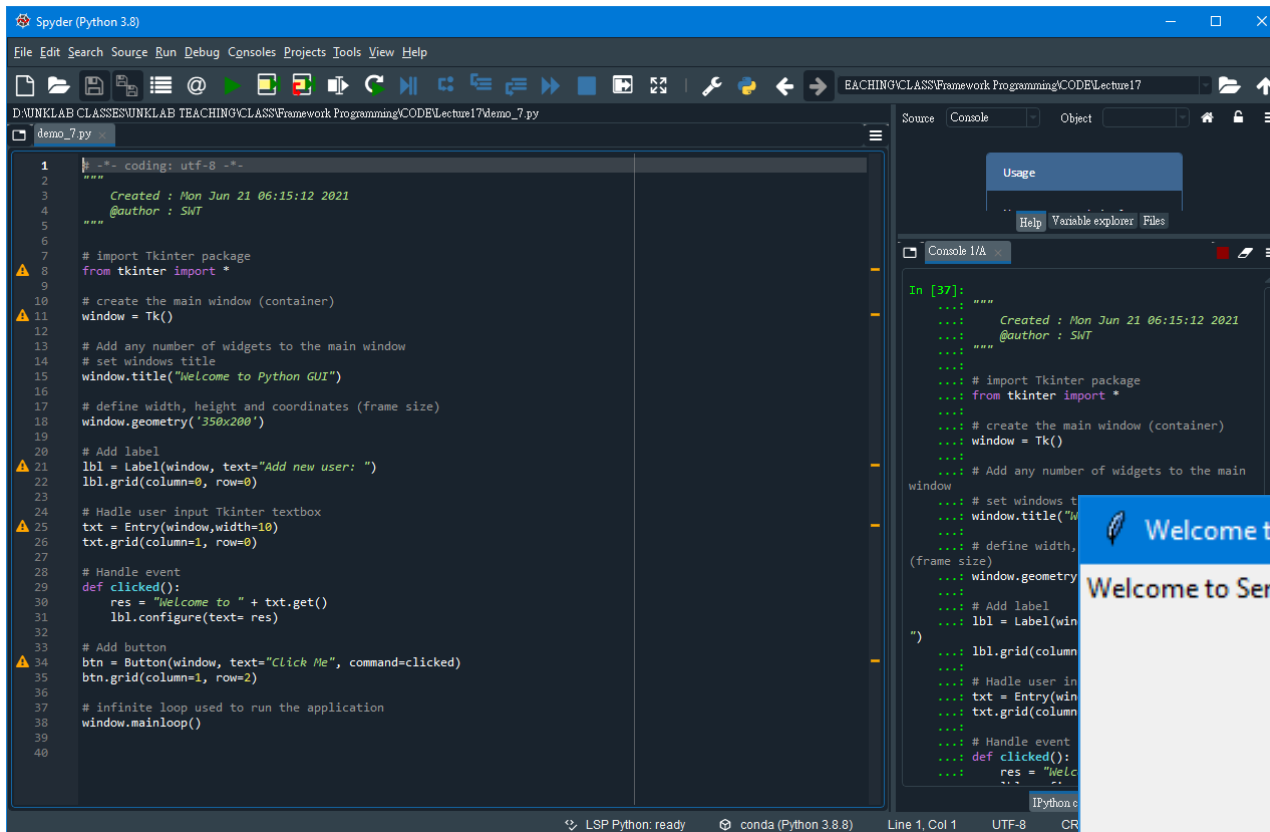


Write a function that we need to execute when the button is clicked.

# Exercise #7

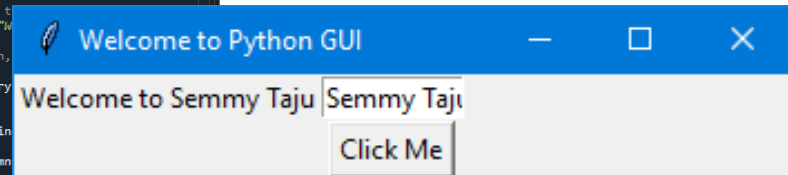
File : *demo\_7.py*

Desc : Get/handle user input “text” using Entry class (Tkinter textbox) or textfield.



```
1  -*- coding: utf-8 -*-
2  """
3  Created : Mon Jun 21 06:15:12 2021
4  @author : SWT
5  """
6
7  # import Tkinter package
8  from tkinter import *
9
10 # create the main window (container)
11 window = Tk()
12
13 # Add any number of widgets to the main window
14 # set windows title
15 window.title("Welcome to Python GUI")
16
17 # define width, height and coordinates (frame size)
18 window.geometry('350x200')
19
20 # Add label
21 lbl = Label(window, text="Add new user: ")
22 lbl.grid(column=0, row=0)
23
24 # Hadle user input Tkinter textbox
25 txt = Entry(window,width=10)
26 txt.grid(column=1, row=0)
27
28 # Handle event
29 def clicked():
30     res = "Welcome to " + txt.get()
31     lbl.configure(text= res)
32
33 # Add button
34 btn = Button(window, text="Click Me", command=clicked)
35 btn.grid(column=1, row=2)
36
37 # infinite loop used to run the application
38 window.mainloop()
39
40
```

Output Program:

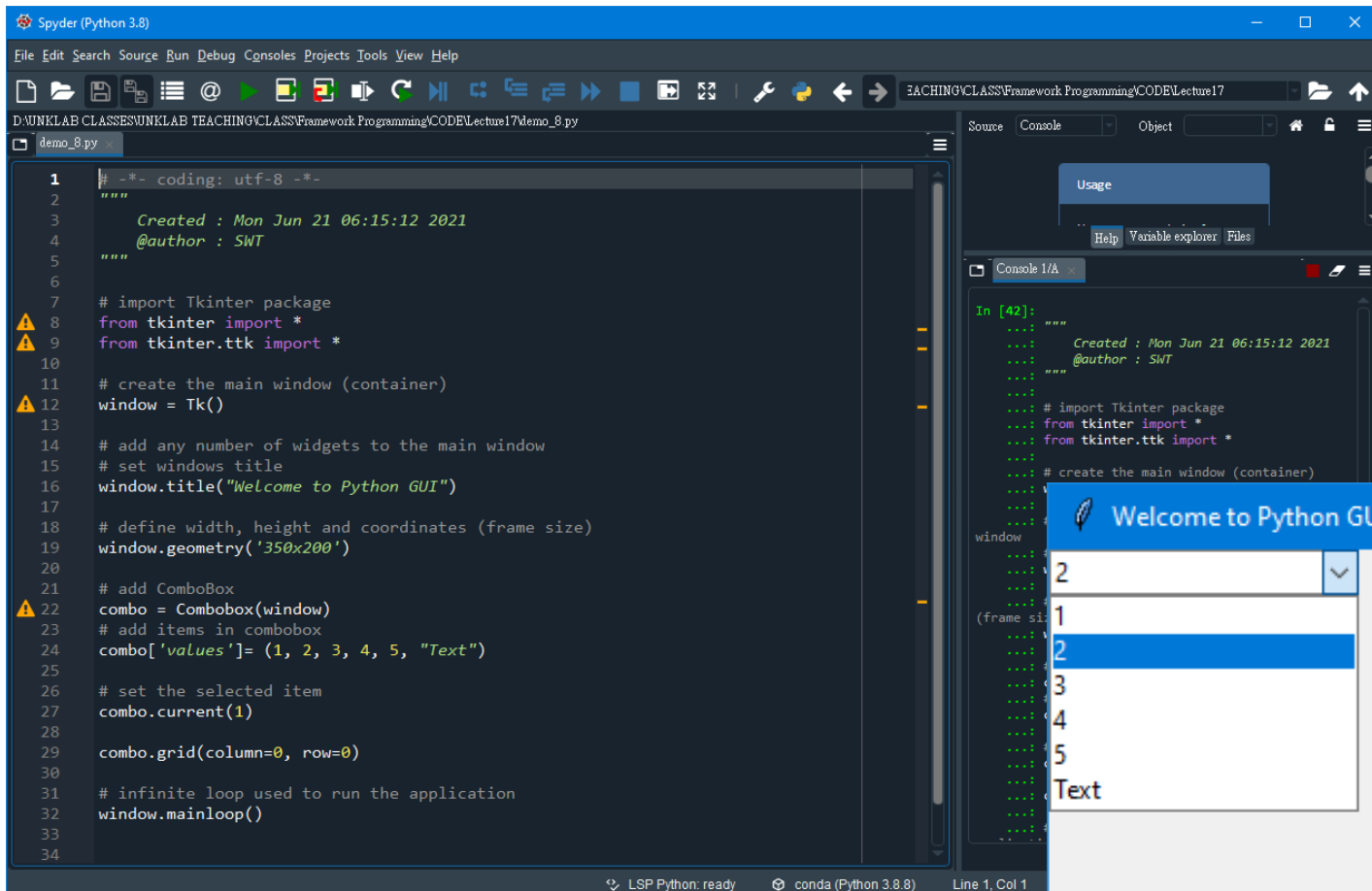


If you click the button and there is a text on the entry widget, it will show “*Welcome to*” concatenated with the entered *text*.

# Exercise #8

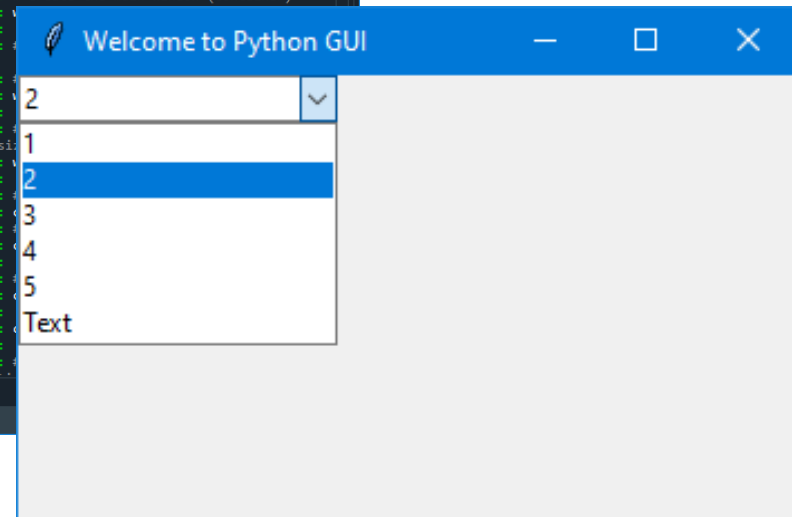
File : *demo\_8.py*

Desc : Add a combobox widget



```
1  -*- coding: utf-8 -*-
2  """
3      Created : Mon Jun 21 06:15:12 2021
4      @author : SWT
5  """
6
7  # import Tkinter package
8  from tkinter import *
9  from tkinter.ttk import *
10
11 # create the main window (container)
12 window = Tk()
13
14 # add any number of widgets to the main window
15 # set windows title
16 window.title("Welcome to Python GUI")
17
18 # define width, height and coordinates (frame size)
19 window.geometry('350x200')
20
21 # add ComboBox
22 combo = Combobox(window)
23 # add items in combobox
24 combo['values']= (1, 2, 3, 4, 5, "Text")
25
26 # set the selected item
27 combo.current(1)
28
29 combo.grid(column=0, row=0)
30
31 # infinite loop used to run the application
32 window.mainloop()
33
34
```

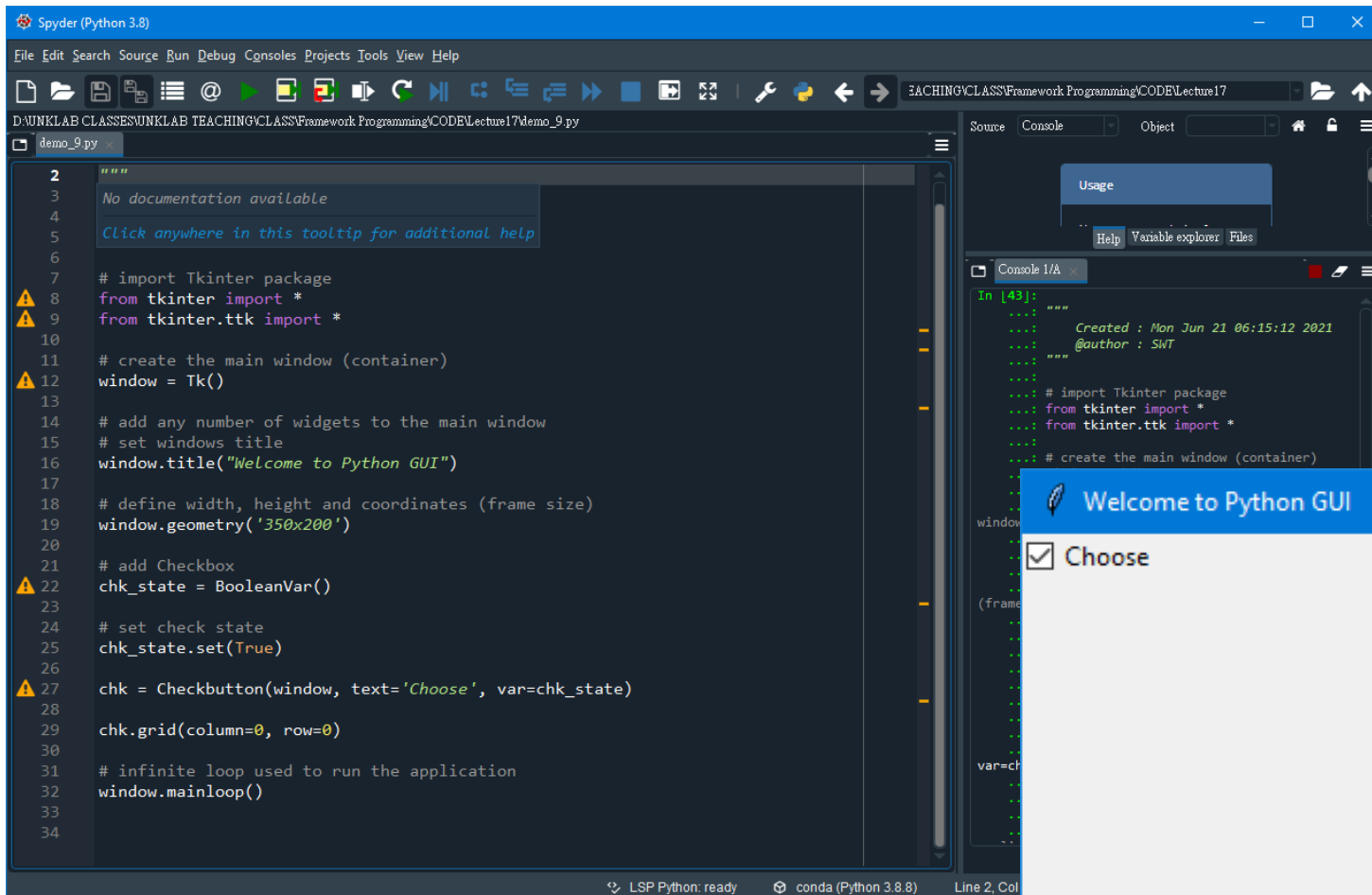
**Output Program:**



# Exercise #9

File : *demo\_9.py*

Desc : Add a Checkbutton widget (Tkinter checkbox)



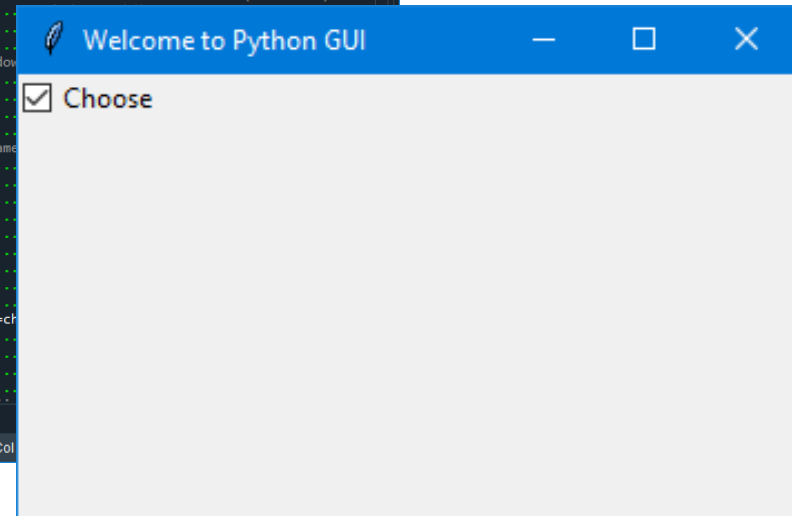
The screenshot shows the Spyder Python IDE with the file `demo_9.py` open. The code defines a Tkinter window titled "Welcome to Python GUI" with a checkbox labeled "Choose". The console shows the execution of the code, including the creation of the window and the checkbox.

```
2 """
3 No documentation available
4
5 Click anywhere in this tooltip for additional help
6
7 # import Tkinter package
8 from tkinter import *
9 from tkinter.ttk import *
10
11 # create the main window (container)
12 window = Tk()
13
14 # add any number of widgets to the main window
15 # set windows title
16 window.title("Welcome to Python GUI")
17
18 # define width, height and coordinates (frame size)
19 window.geometry('350x200')
20
21 # add Checkbox
22 chk_state = BooleanVar()
23
24 # set check state
25 chk_state.set(True)
26
27 chk = Checkbutton(window, text='Choose', var=chk_state)
28
29 chk.grid(column=0, row=0)
30
31 # infinite loop used to run the application
32 window.mainloop()
33
34
```

Console 1/A

```
In [43]:
...: """
...: Created : Mon Jun 21 06:15:12 2021
...: @author : SWT
...: """
...:
...: # import Tkinter package
...: from tkinter import *
...: from tkinter.ttk import *
...:
...: # create the main window (container)
...: window = Tk()
...:
...: # add Checkbox
...: chk_state = BooleanVar()
...:
...: # set check state
...: chk_state.set(True)
...:
...: chk = Checkbutton(window, text='Choose', var=chk_state)
...:
...: chk.grid(column=0, row=0)
...:
...: # infinite loop used to run the application
...: window.mainloop()
...:
```

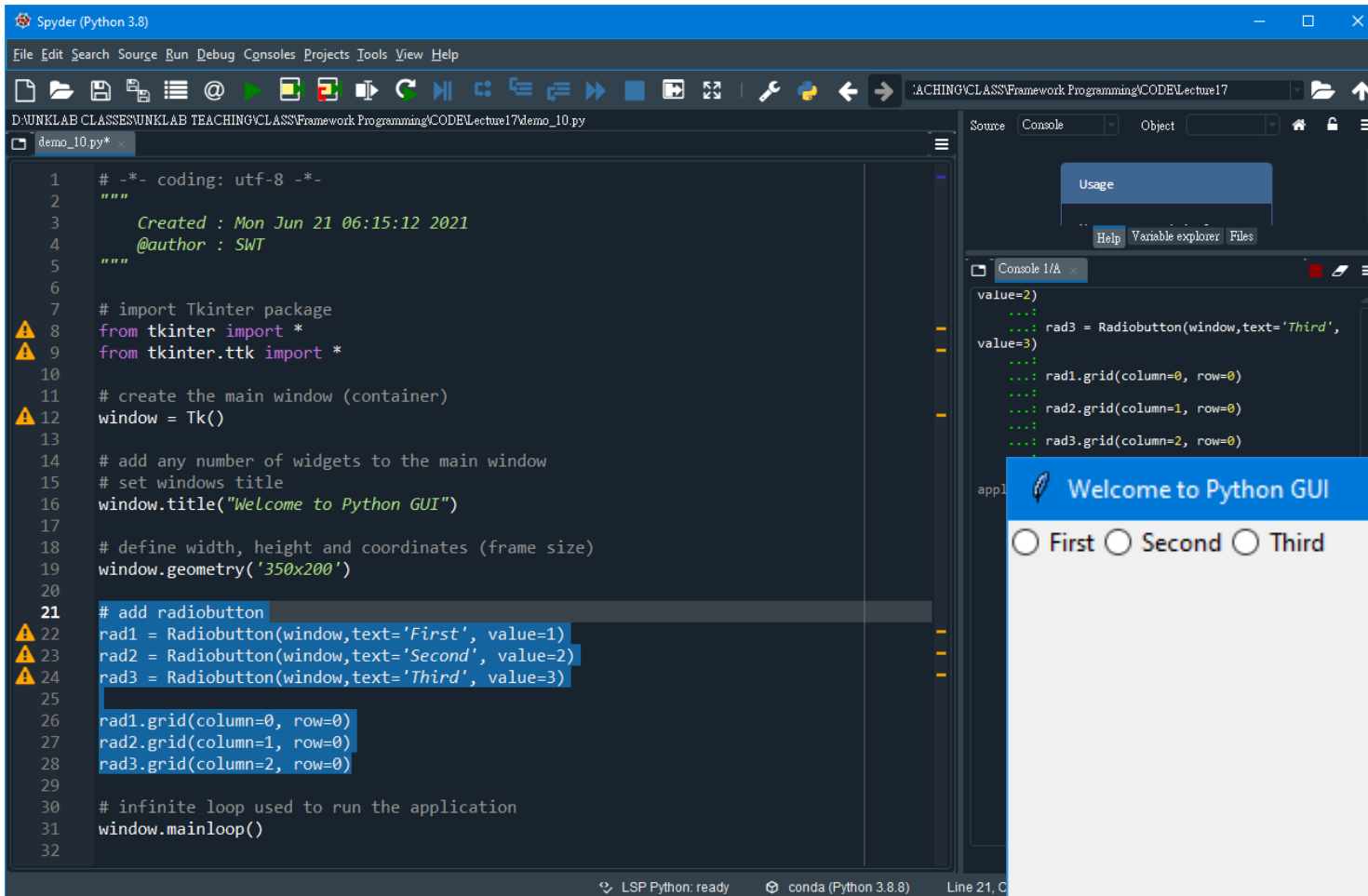
**Output Program:**



# Exercise #10

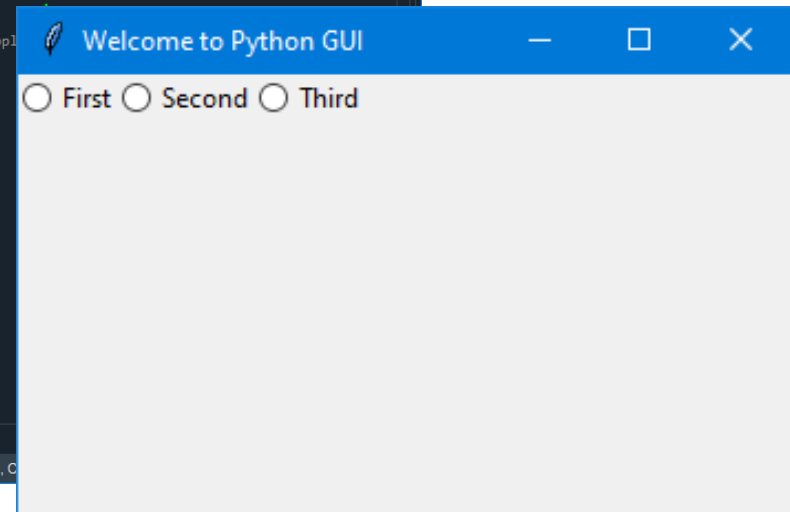
File : *demo\_10.py*

Desc : Add radio buttons widgets



```
1  #-*- coding: utf-8 -*-
2
3      Created : Mon Jun 21 06:15:12 2021
4      @author : SWT
5
6
7  # import Tkinter package
8  from tkinter import *
9  from tkinter.ttk import *
10
11 # create the main window (container)
12 window = Tk()
13
14 # add any number of widgets to the main window
15 # set windows title
16 window.title("Welcome to Python GUI")
17
18 # define width, height and coordinates (frame size)
19 window.geometry('350x200')
20
21 # add radiobutton
22 rad1 = Radiobutton(window,text='First', value=1)
23 rad2 = Radiobutton(window,text='Second', value=2)
24 rad3 = Radiobutton(window,text='Third', value=3)
25
26 rad1.grid(column=0, row=0)
27 rad2.grid(column=1, row=0)
28 rad3.grid(column=2, row=0)
29
30 # infinite loop used to run the application
31 window.mainloop()
32
```

**Output Program:**

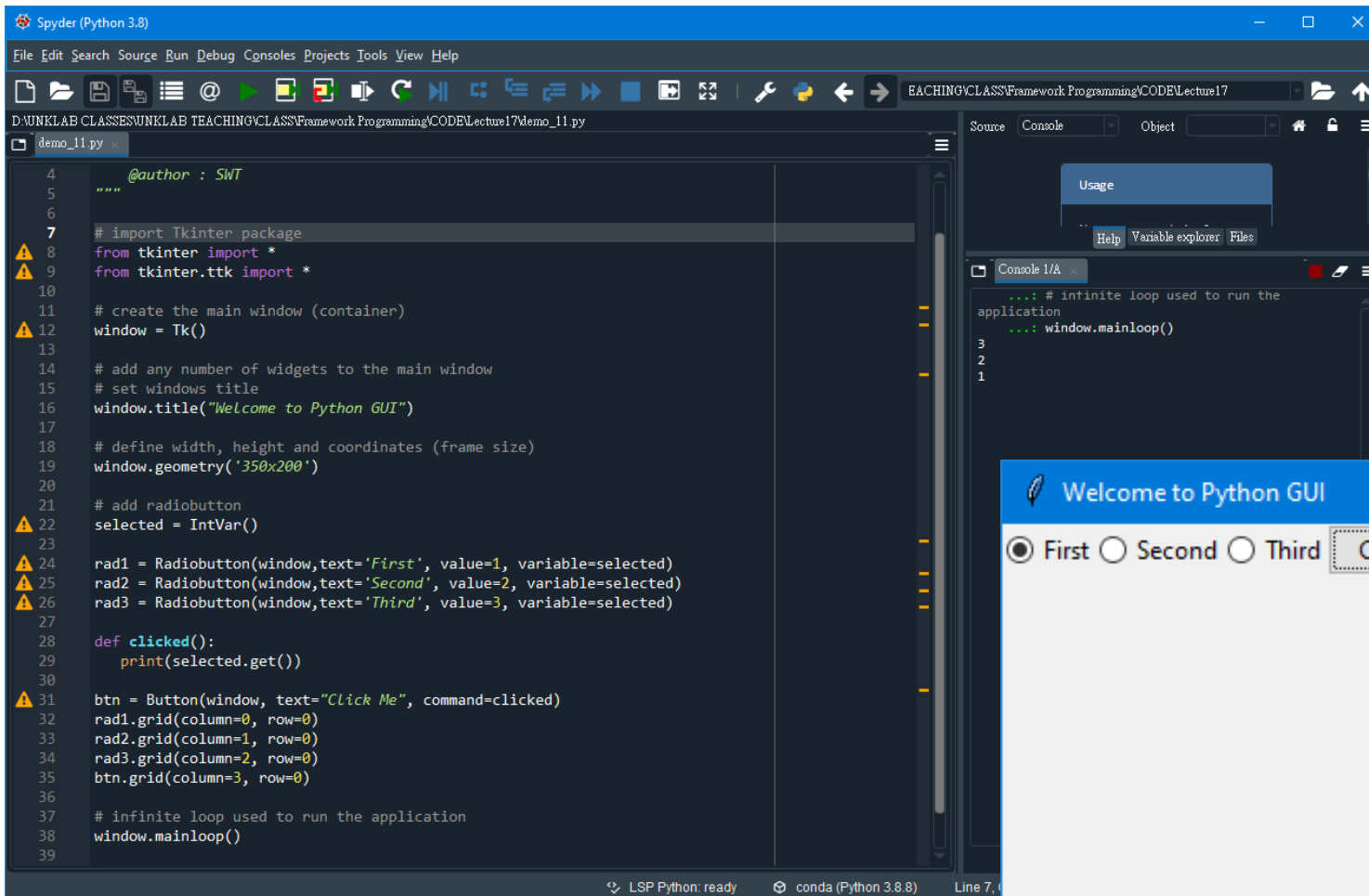




# Exercise #11

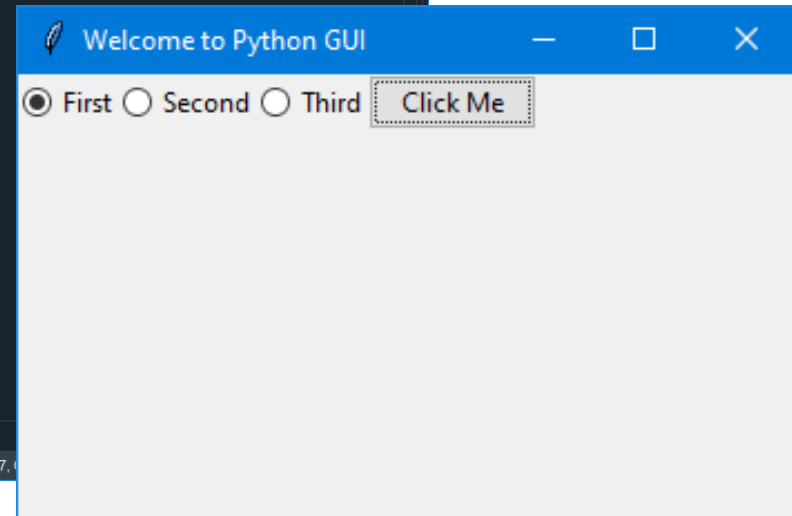
File : *demo\_11.py*

Desc : Get radio button value (selected radio button)



```
4  @author : SWT
5  """
6
7  # import Tkinter package
8  from tkinter import *
9  from tkinter.ttk import *
10
11 # create the main window (container)
12 window = Tk()
13
14 # add any number of widgets to the main window
15 # set windows title
16 window.title("Welcome to Python GUI")
17
18 # define width, height and coordinates (frame size)
19 window.geometry('350x200')
20
21 # add radiobutton
22 selected = IntVar()
23
24 rad1 = Radiobutton(window,text='First', value=1, variable=selected)
25 rad2 = Radiobutton(window,text='Second', value=2, variable=selected)
26 rad3 = Radiobutton(window,text='Third', value=3, variable=selected)
27
28 def clicked():
29     print(selected.get())
30
31 btn = Button(window, text="Click Me", command=clicked)
32 rad1.grid(column=0, row=0)
33 rad2.grid(column=1, row=0)
34 rad3.grid(column=2, row=0)
35 btn.grid(column=3, row=0)
36
37 # infinite loop used to run the application
38 window.mainloop()
39
```

**Output Program:**



# Exercise #12

File : *demo\_12.py*

Desc : Create a MessageBox

The screenshot displays the Spyder Python IDE interface. The main editor window shows a Python script named `demo_12.py` with the following code:

```
1  -*- coding: utf-8 -*-
2  """
3      Created : Mon Jun 21 06:15:12 2021
4      @author : SWT
5  """
6
7  # import Tkinter package
8  from tkinter import *
9  from tkinter.ttk import *
10
11 # create the main window (container)
12 window = Tk()
13
14 # add any number of widgets to the main window
15 # set windows title
16 window.title("Welcome to Python GUI")
17
18 # define width, height and coordinates (frame size)
19 window.geometry('350x200')
20
21 # add Message dialog
22 def clicked():
23     messagebox.showinfo('Message title', 'Message content')
24
25 # add button
26 btn = Button(window, text='Click here', command=clicked)
27 btn.grid(column=0, row=0)
28
29 # infinite loop used to run the application
30 window.mainloop()
```

The right-hand pane shows the IPython console with the following output:

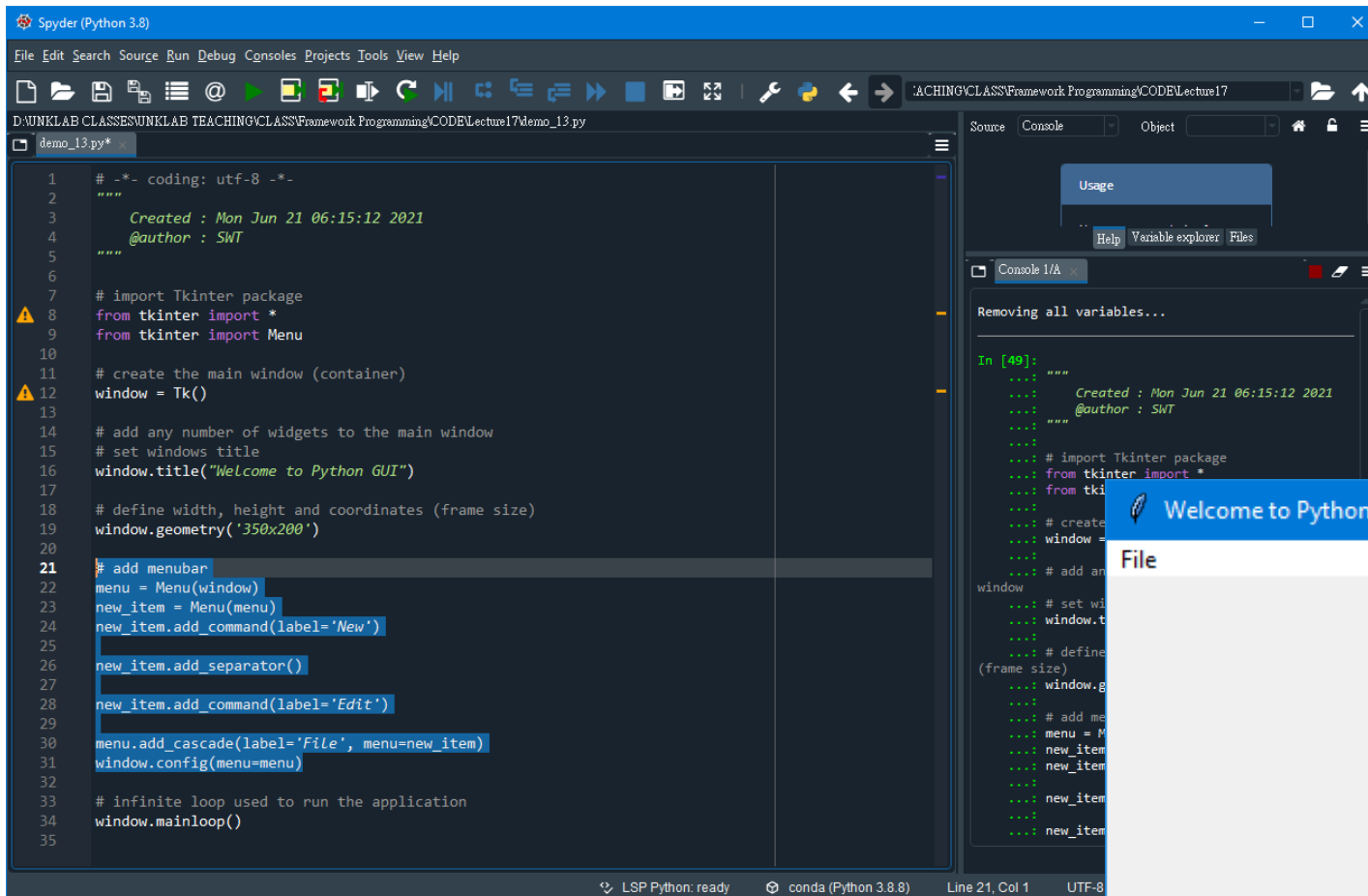
```
...: window.title("Welcome to Python GUI")
...:
...: # define width, height and coordinates (frame size)
...: window.geometry('350x200')
...:
...: # add Message dialog
...: def clicked():
...:     messagebox.showinfo('Message title', 'Message content')
...:
...: # add button
...: btn = Button(window, text='Click here', command=clicked)
...: btn.grid(column=0, row=0)
...:
...: # infinite loop used to run the application
...: window.mainloop()
```

Overlaid on the IDE is a screenshot of the running GUI. It features a main window titled "Welcome to Python GUI" with a "Click here" button. Upon clicking the button, a message box titled "Message title" appears, displaying "Message content" with an information icon and an "OK" button.

# Exercise #13

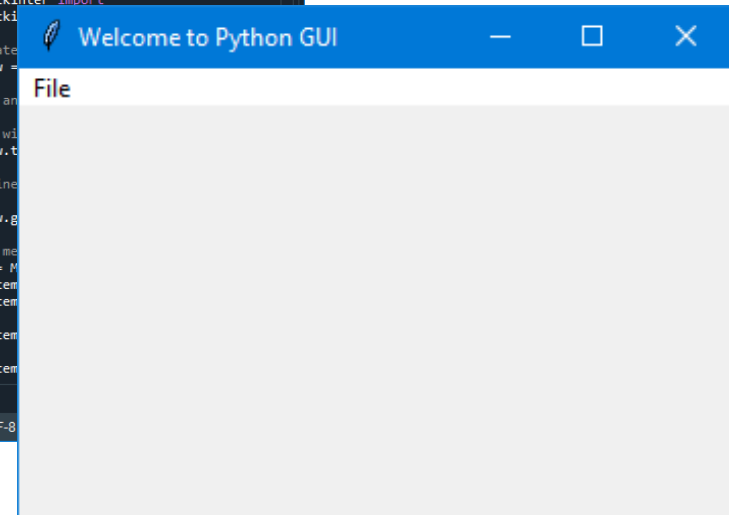
File : *demo\_13.py*

Desc : Add a Menu bar



```
1  # -*- coding: utf-8 -*-
2
3  """
4      Created : Mon Jun 21 06:15:12 2021
5      @author : SWT
6  """
7
8  # import Tkinter package
9  from tkinter import *
10 from tkinter import Menu
11
12 # create the main window (container)
13 window = Tk()
14
15 # add any number of widgets to the main window
16 # set windows title
17 window.title("Welcome to Python GUI")
18
19 # define width, height and coordinates (frame size)
20 window.geometry('350x200')
21
22 # add menubar
23 menu = Menu(window)
24 new_item = Menu(menu)
25 new_item.add_command(label='New')
26 new_item.add_separator()
27 new_item.add_command(label='Edit')
28 menu.add_cascade(label='File', menu=new_item)
29 window.config(menu=menu)
30
31 # infinite loop used to run the application
32 window.mainloop()
33
34
35
```

**Output Program:**



# ModuleNotFoundError?

## ModuleNotFoundError: No module named 'Image'

```
Anaconda Prompt (Anaconda3) - pip install image

(semmy_env) C:\Users\NL>pip install image
Collecting image
  Downloading image-1.5.33.tar.gz (15 kB)
Requirement already satisfied: pillow in c:\programdata\anaconda3\lib\site-packages (from image) (8.2.0)
Collecting django
  Downloading Django-3.2.4-py3-none-any.whl (7.9 MB)
  | 7.5 MB 2.2 MB/s eta 0:00:01
```

```
Anaconda Prompt (Anaconda3)

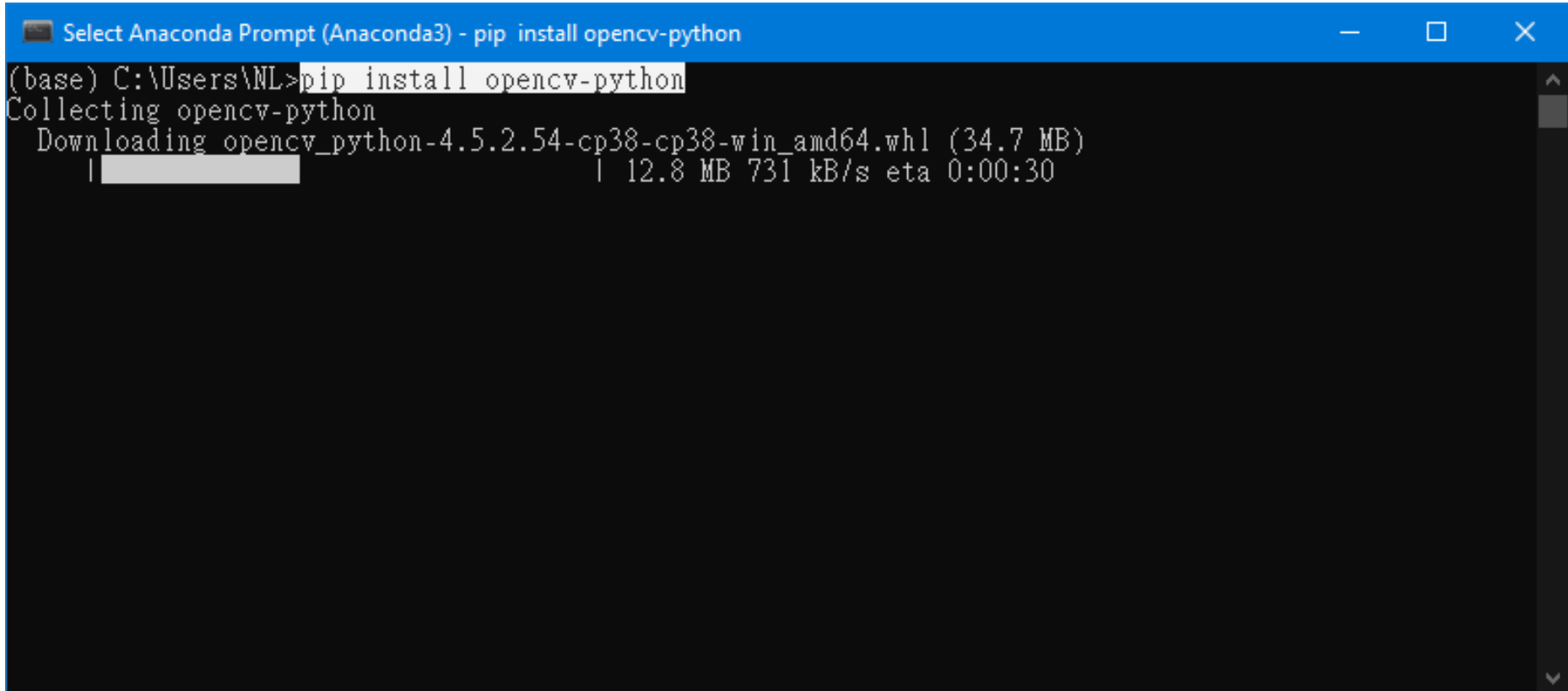
(semmy_env) C:\Users\NL>pip install Pillow --user
Requirement already satisfied: Pillow in c:\programdata\anaconda3\lib\site-packages (8.2.0)

(semmy_env) C:\Users\NL>pip install Pillow
Requirement already satisfied: Pillow in c:\programdata\anaconda3\lib\site-packages (8.2.0)

(semmy_env) C:\Users\NL>
```

# Install OpenCV Package

Install OpenCV on the platform via *pip*:



```
Select Anaconda Prompt (Anaconda3) - pip install opencv-python

(base) C:\Users\NL>pip install opencv-python
Collecting opencv-python
  Downloading opencv_python-4.5.2.54-cp38-cp38-win_amd64.whl (34.7 MB)
    | 12.8 MB 731 kB/s eta 0:00:30
```

# Switch Between Environments in Anaconda Navigator

The screenshot displays the Anaconda Navigator desktop application. The interface includes a top menu bar with 'File' and 'Help', a left sidebar with navigation options like 'Home', 'Environments', 'Learning', and 'Community', and a main workspace area. In the workspace, a dropdown menu is open, showing the current environment 'semmy\_env' and two other options: 'base (root)' and 'semmy\_env' (marked with a green checkmark). This dropdown is highlighted with a red rectangle. Below the dropdown, a grid of application tiles is visible, including Datalore, IBM Watson Studio Cloud, CMD.exe Prompt, Glueviz, JupyterLab, Jupyter Notebook, Orange 3, and Powershell Prompt. Each tile contains a description and a button to either 'Launch' or 'Install' the application.

Applications on **semmy\_env** Channels Refresh

- Datalore**  
Online Data Analysis Tool with smart coding assistance by JetBrains. Edit and run your Python notebooks in the cloud and share them with your team.  
[Launch](#)
- IBM Watson Studio Cloud**  
IBM Watson Studio Cloud provides you the tools to analyze and visualize data, to cleanse and shape data, to create and train machine learning models. Prepare data and build models, using open source data science tools or visual modeling.  
[Launch](#)
- CMD.exe Prompt**  
0.1.1  
Run a cmd.exe terminal with your current environment from Navigator activated  
[Install](#)
- Glueviz**  
1.0.0  
Multidimensional data visualization across files. Explore relationships within and among related datasets.  
[Install](#)
- JupyterLab**  
3.0.14  
An extensible environment for interactive and reproducible computing, based on the Jupyter Notebook and Architecture.
- Jupyter Notebook**  
6.4.0  
Web-based, interactive computing notebook environment. Edit and run human-readable docs while describing the data analysis.
- Orange 3**  
3.26.0  
Component based data mining framework. Data visualization and data analysis for novice and expert. Interactive workflows with a large toolbox.
- Powershell Prompt**  
0.0.1  
Run a Powershell terminal with your current environment from Navigator activated

# Activate an Anaconda Environment

Anaconda Prompt (Anaconda3) - spyder

```
(base) C:\Users\NL>conda activate semmy_env
```

```
(semmy_env) C:\Users\NL>spyder
```

# Exercise #14

File : *demo\_14.py*

Desc : Pthon3, tkinter (GUI) and openCV to develop a simple Face Detection System

The image displays the Spyder Python IDE interface with a Python 3.8 environment. The main editor window shows the code for `demo_14.py`, which implements a face detection system using OpenCV and Tkinter. The code includes imports for `sys`, `importlib`, `cv2`, `tkinter`, `tkinter.messagebox`, and `filedialog`. It sets the current working directory to `D:\UNKLAB CLASSES\UNKLAB TEACHING\CLASS\Framework Programming\CODE\Lecture17`. The `selectPath()` function uses `filedialog.askopenfilename()` to select an image file. The `detectFaceImage()` function uses `cv2.CascadeClassifier` to detect faces in the selected image. It converts the image to grayscale and uses `detectMultiScale` to find faces. If faces are detected, it prints the count and shows a message box. If no faces are detected, it shows an error message. The GUI is a simple Tkinter window titled "Face Detection System" with a text field for the image path and a "Detect Face" button. The console shows the output: "System recognized 2 faces." and "System recognized 2 face". Below the code, there are two windows showing the detected faces: "Recognized 2 faces" and "Recognized 2 faces".

```
1  -*- coding: utf-8 -*-
2  import sys
3  import importlib
4  import cv2
5  import tkinter as tk
6  import tkinter.messagebox
7  from tkinter import filedialog
8
9  # Set current Working Directory
10 work_dir = "D:\UNKLAB CLASSES\UNKLAB TEACHING\CLASS\Framework Programming\CODE\Lecture17";
11
12 # Function to read image path
13 def selectPath():
14     global path_
15     path_ = filedialog.askopenfilename()
16     path.set(path_)
17
18 # Handle Open CV when click button
19 def detectFaceImage():
20     try:
21         # github Obtaining trained face parameter data
22         face_cascade = cv2.CascadeClassifier('{0}/cascades/haarcascade_frontalface_default.xml'.format(work_dir))
23
24         # Read pictures
25         image = cv2.imread(path_)
26
27         # Convert to Grayscale
28         gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
29
30         # Detecting faces in pictures
31         faces = face_cascade.detectMultiScale( gray, scaleFactor=1.15, minNeighbors=5, minSize=(5, 5), flags=cv2.IMREAD_GRAYSCALE)
32
33         if(len(faces)==0):
34             tkinter.messagebox.showerror('Error Message', 'No face recognized, please choose a clearer picture.')
35
36         print("System recognized {0} faces.".format(len(faces)))
37
38         # faces [x-coordinate, p-coordinate, width, and length of the upper left corner]
39         for (x, y, w, h) in faces:
40             cv2.rectangle(image, (x,y), (x+w,y+w), (255,255,0),1)
41             if (len(faces) > 0):
42                 cv2.imshow("Recognized {0} faces".format(len(faces)), image)
43
44         # Key press handling
45         cv2.waitKey(0)
46     except:
47         tkinter.messagebox.showerror('Error Message', 'Please select the correct picture file.')
```

Face Detection System

Please select a picture path

Select image file: C:/Users/NL/Pictures/t Detect Face

System recognized 2 faces

Recognized 2 faces



# Exercise #14

File : *demo\_14.py*

Desc : Python3, tkinter (GUI) and openCV to develop a simple Face Detection System

```
# Design Python GUI
#importlib.reload(sys)

window = tk.Tk()
window.title('Face Detection System')

fm1 = tk.Frame(window)
fm2 = tk.Frame(window)

path = tk.StringVar()

Ltop=tk.Label(fm1,text="Please select a picture path")
B1=tk.Button(fm2, text = "Select image file: ", command = selectPath)
E1=tk.Entry(fm2, textvariable = path, bd=5)
B2=tk.Button(fm2, text = "Detect Face", command = detectFaceImage)

Ltop.pack(side = tk.TOP)
B1.pack(side = tk.LEFT)
E1.pack(side = tk.LEFT)
B2.pack(side = tk.LEFT)

fm1.pack(side = tk.TOP)
fm2.pack(side = tk.TOP)

#Get the screen width & height
sw = window.winfo_screenwidth()
sh = window.winfo_screenheight()

ww = 300
wh = 100

#window width and height 100
x = (sw-ww) / 2
y = (sh-wh) / 3
window.geometry("%dx%d+%d+%d" %(ww,wh,x,y))

# infinite loop used to run the application
window.mainloop()
```

A decorative border of green leaves and branches at the top of the page.

# 2 - Install **Flask** on Windows

*[Web Application Framework with Python]*

A decorative border of green leaves and branches at the bottom of the page.

# What is Flask?

Flask is a ***web application framework*** written in *Python*. It is developed by Armin Ronacher, who leads an international group of Python enthusiasts named Pocco. Flask is based on the Werkzeug WSGI toolkit and Jinja2 template engine. Both are Pocco projects.

## 1) WSGI

Web Server Gateway Interface (WSGI) has been adopted as a standard for Python web application development. WSGI is a specification for a universal interface between the web server and the web applications.

## 2) Werkzeug

It is a WSGI toolkit, which implements requests, response objects, and other utility functions. This enables building a web framework on top of it. The Flask framework uses Werkzeug as one of its bases.

## 3) Jinja2

Jinja2 is a popular templating engine for Python. A web templating system combines a template with a certain data source to render dynamic web pages.

# Prerequisites (*prasyarat*)

- ❑ Installed Python 2.7 or Python 3.5 and newer.
- ❑ Command-line interface (CLI) with administrator privileges.

# Step 1: Install Virtual Environment

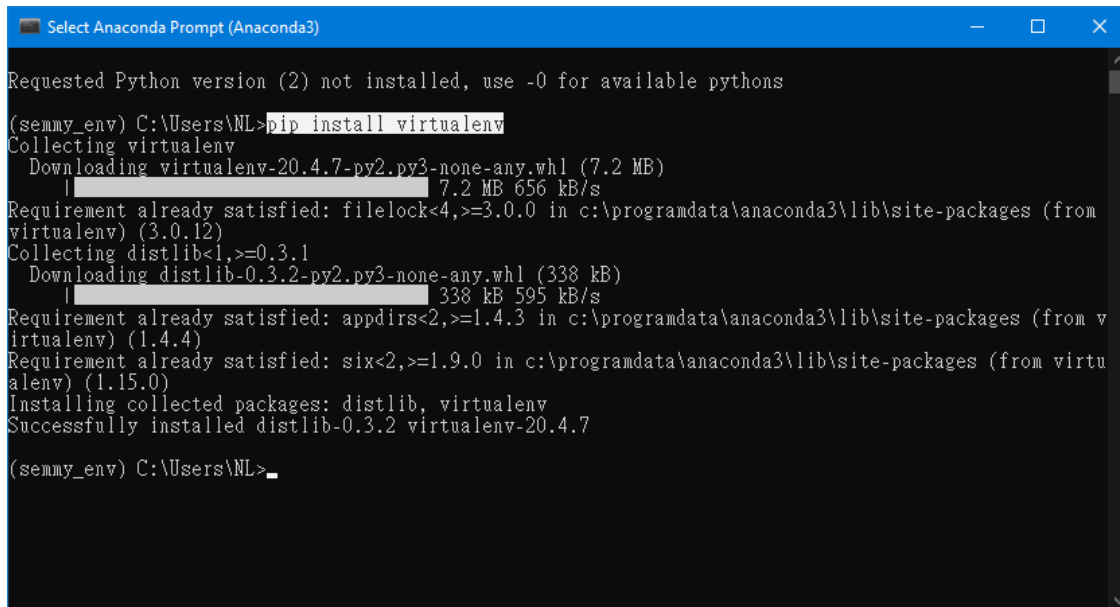
Install Flask in a virtual environment to avoid problems with conflicting libraries.

Check Python version before starting:

- 1) Python 3 comes with a virtual environment module called venv **preinstalled**. If you have Python 3 installed, **skip to Step 2**.
- 2) Python 2 users must install the virtualenv module. If you have Python 2, **follow the instructions outlined in Step 1**.

Install *virtualenv* on Windows:

1. Open the command line with administrator privileges.
2. Use pip to install virtualenv on Windows: ***pip install virtualenv***

A screenshot of an Anaconda Prompt window titled "Select Anaconda Prompt (Anaconda3)". The terminal shows the command "pip install virtualenv" being executed. The output indicates that the requested Python version (2) is not installed, but the installation proceeds. It shows the collection and downloading of virtualenv-20.4.7 (7.2 MB) and distlib-0.3.2 (338 kB). It also shows that requirements for filelock, appdirs, and six are already satisfied. Finally, it states "Successfully installed distlib-0.3.2 virtualenv-20.4.7".

```
Select Anaconda Prompt (Anaconda3)

Requested Python version (2) not installed, use -0 for available pythons

(semmy_env) C:\Users\NL>pip install virtualenv
Collecting virtualenv
  Downloading virtualenv-20.4.7-py2.py3-none-any.whl (7.2 MB)
    | 7.2 MB 656 kB/s
Requirement already satisfied: filelock<4,>=3.0.0 in c:\programdata\anaconda3\lib\site-packages (from v
irtualenv) (3.0.12)
Collecting distlib<1,>=0.3.1
  Downloading distlib-0.3.2-py2.py3-none-any.whl (338 kB)
    | 338 kB 595 kB/s
Requirement already satisfied: appdirs<2,>=1.4.3 in c:\programdata\anaconda3\lib\site-packages (from v
irtualenv) (1.4.4)
Requirement already satisfied: six<2,>=1.9.0 in c:\programdata\anaconda3\lib\site-packages (from virtu
alenv) (1.15.0)
Installing collected packages: distlib, virtualenv
Successfully installed distlib-0.3.2 virtualenv-20.4.7

(semmy_env) C:\Users\NL>
```

## Step 2: Create an Environment

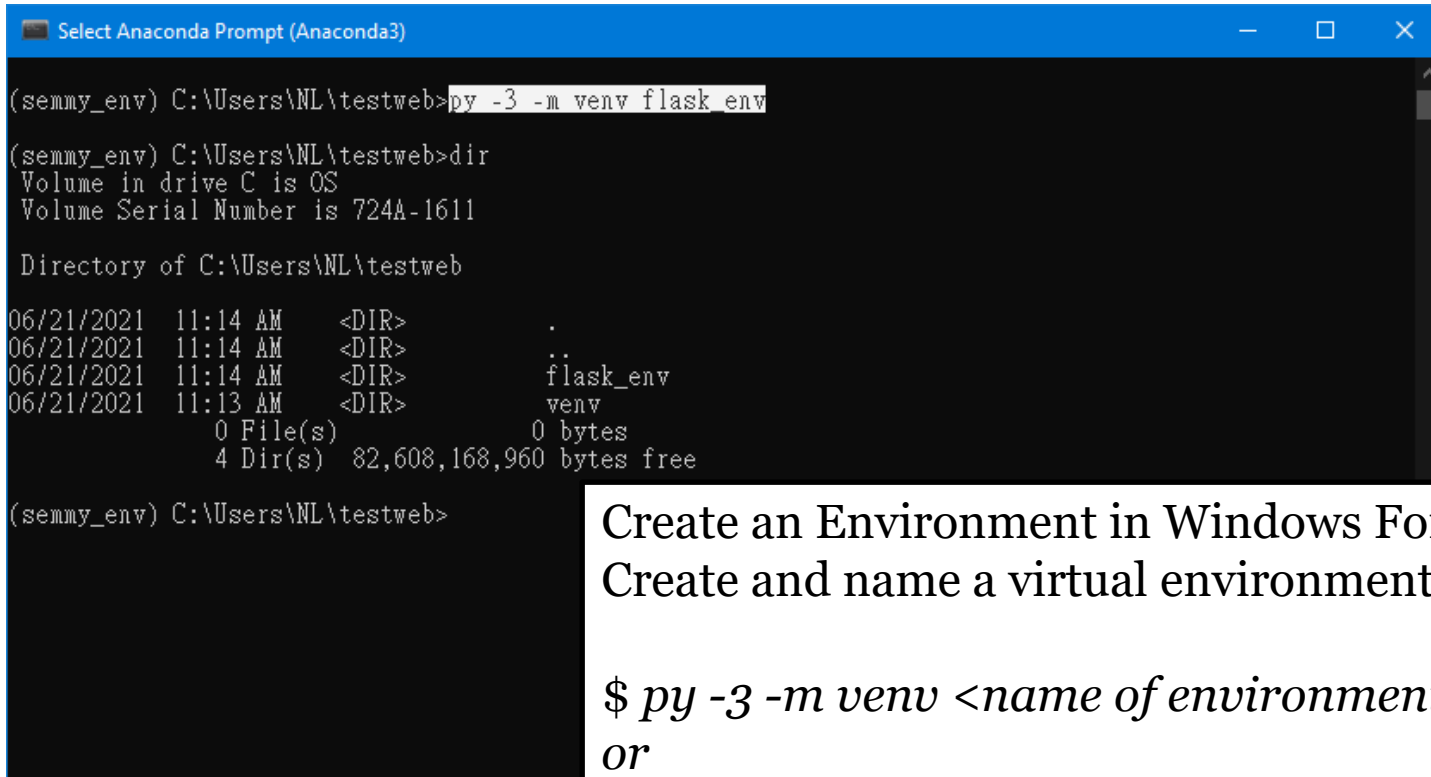
- 1) Make a separate directory for your project:

*\$ mkdir testweb*

- 2) Move into the directory:

*\$ cd testweb*

Within the directory, create the virtual environment for Flask. When you create the environment, a new folder appears in your project directory with the environment's name.



```
Select Anaconda Prompt (Anaconda3)

(semmy_env) C:\Users\NL\testweb>py -3 -m venv flask_env

(semmy_env) C:\Users\NL\testweb>dir
Volume in drive C is OS
Volume Serial Number is 724A-1611

Directory of C:\Users\NL\testweb

06/21/2021  11:14 AM    <DIR>          .
06/21/2021  11:14 AM    <DIR>          ..
06/21/2021  11:14 AM    <DIR>          flask_env
06/21/2021  11:13 AM    <DIR>          venv
               0 File(s)              0 bytes
               4 Dir(s)  82,608,168,960 bytes free

(semmy_env) C:\Users\NL\testweb>
```

Create an Environment in Windows For Python 3:  
Create and name a virtual environment in Python 3 with:

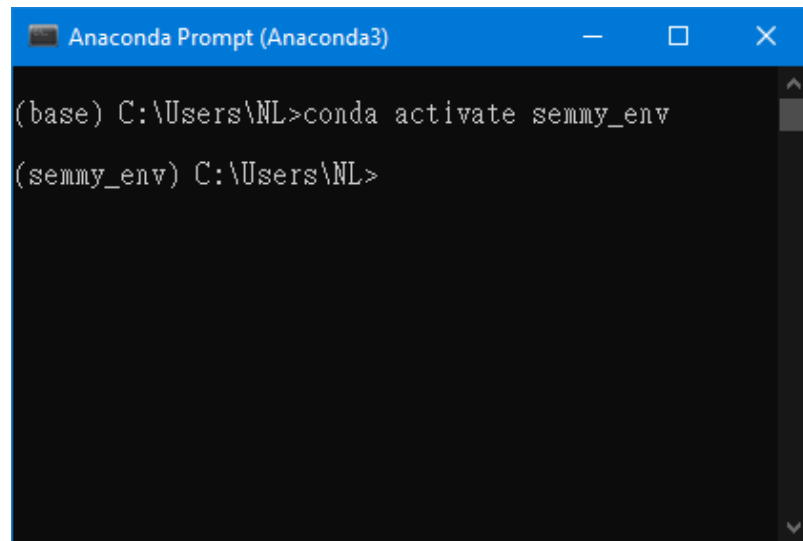
*\$ py -3 -m venv <name of environment>*

*or*

*\$ python-m venv <name of environment>*

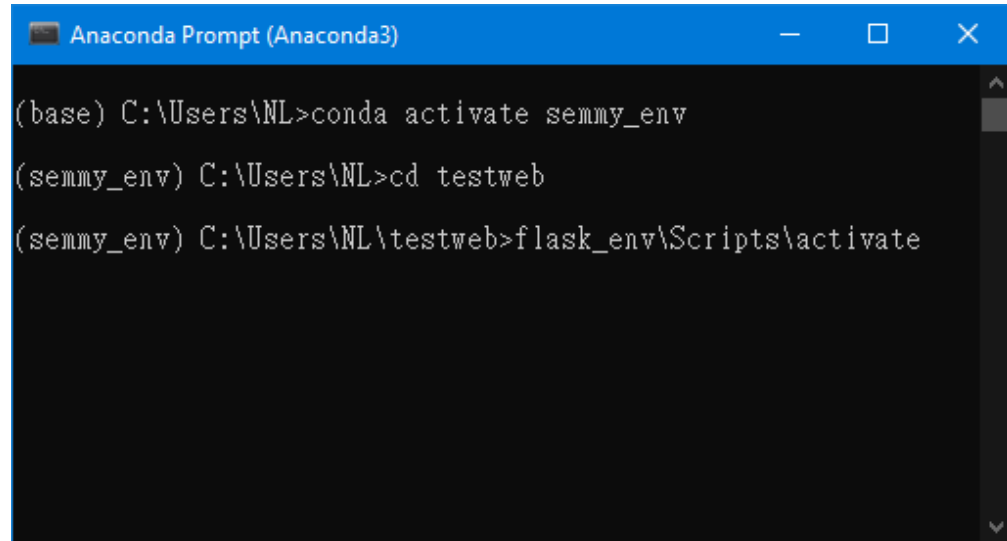
# Step 3: Activate the Environment

## 1. Activate conda Environment on Windows:



```
Anaconda Prompt (Anaconda3)
(base) C:\Users\NL>conda activate semmy_env
(semmy_env) C:\Users\NL>
```

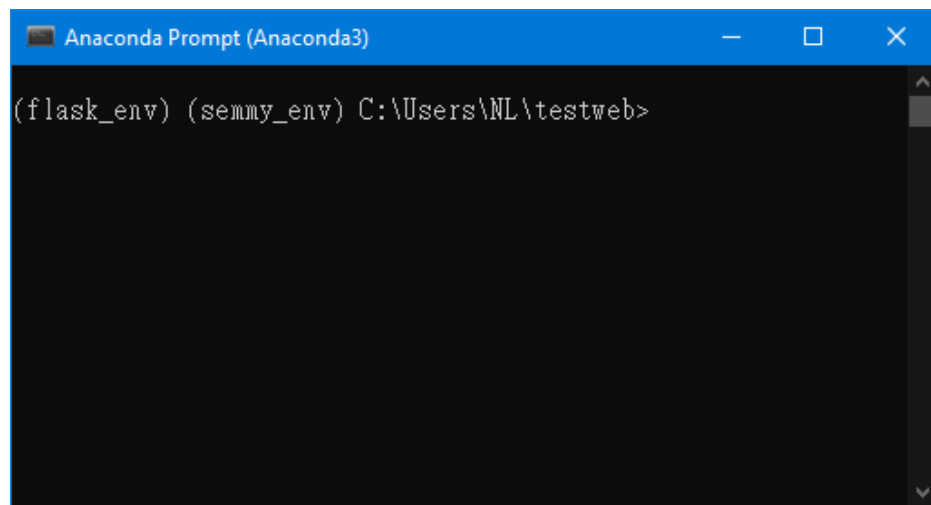
## 2. Activate the Virtual Environment for flask



```
Anaconda Prompt (Anaconda3)
(base) C:\Users\NL>conda activate semmy_env
(semmy_env) C:\Users\NL>cd testweb
(semmy_env) C:\Users\NL\testweb>flask_env\Scripts\activate
```

For Windows, activate the virtual environment.

## 3. Activated Virtual Environment

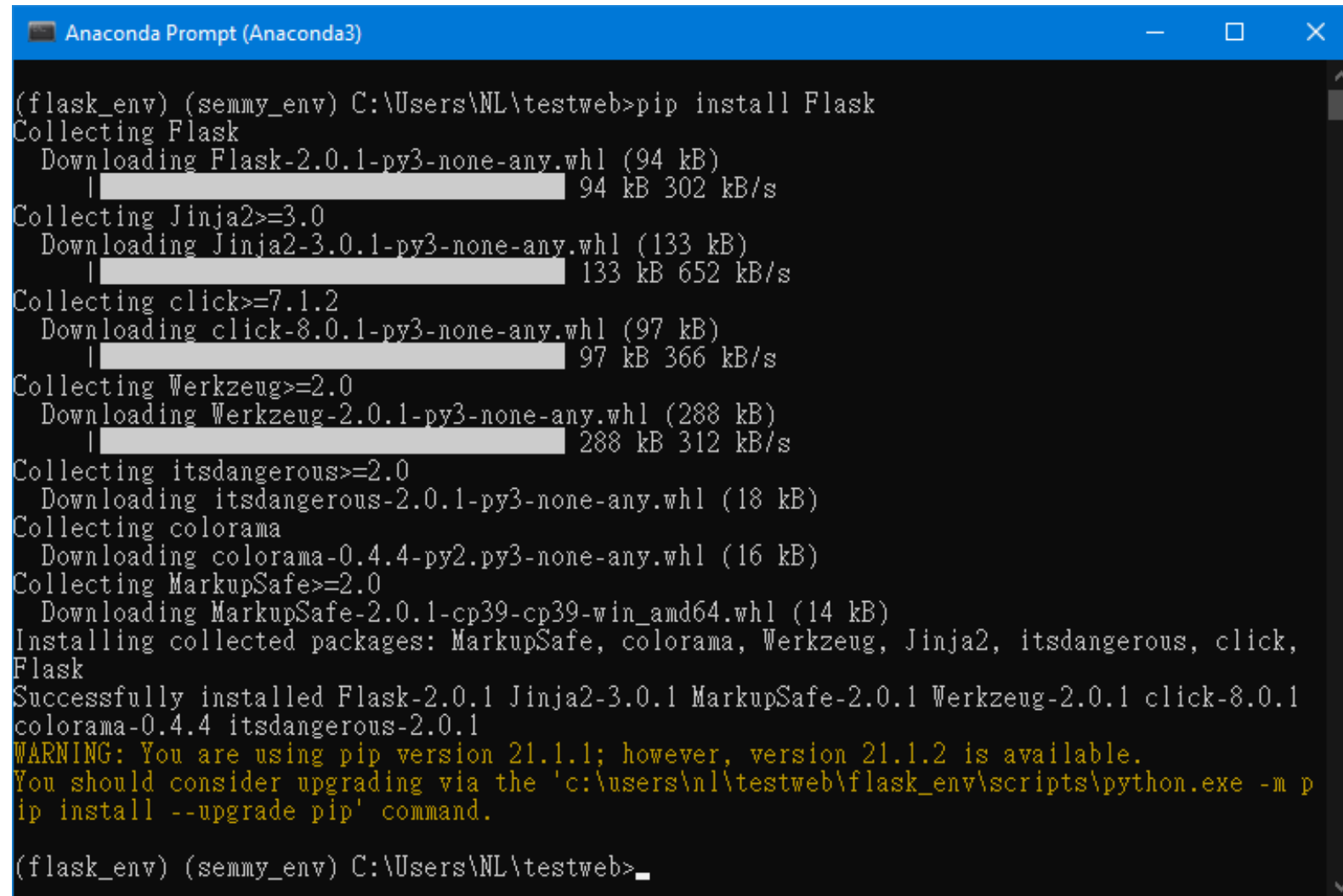


```
Anaconda Prompt (Anaconda3)
(flask_env) (semmy_env) C:\Users\NL\testweb>
```

# Step 4: Install Flask

Install Flask within the activated environment using *pip*:

\$ *pip install Flask*



```
Anaconda Prompt (Anaconda3)

(flask_env) (semmy_env) C:\Users\NL\testweb>pip install Flask
Collecting Flask
  Downloading Flask-2.0.1-py3-none-any.whl (94 kB)
    | ████████████████████████████████████████ 94 kB 302 kB/s
Collecting Jinja2>=3.0
  Downloading Jinja2-3.0.1-py3-none-any.whl (133 kB)
    | ████████████████████████████████████████ 133 kB 652 kB/s
Collecting click>=7.1.2
  Downloading click-8.0.1-py3-none-any.whl (97 kB)
    | ████████████████████████████████████████ 97 kB 366 kB/s
Collecting Werkzeug>=2.0
  Downloading Werkzeug-2.0.1-py3-none-any.whl (288 kB)
    | ████████████████████████████████████████ 288 kB 312 kB/s
Collecting itsdangerous>=2.0
  Downloading itsdangerous-2.0.1-py3-none-any.whl (18 kB)
Collecting colorama
  Downloading colorama-0.4.4-py2.py3-none-any.whl (16 kB)
Collecting MarkupSafe>=2.0
  Downloading MarkupSafe-2.0.1-cp39-cp39-win_amd64.whl (14 kB)
Installing collected packages: MarkupSafe, colorama, Werkzeug, Jinja2, itsdangerous, click,
Flask
Successfully installed Flask-2.0.1 Jinja2-3.0.1 MarkupSafe-2.0.1 Werkzeug-2.0.1 click-8.0.1
colorama-0.4.4 itsdangerous-2.0.1
WARNING: You are using pip version 21.1.1; however, version 21.1.2 is available.
You should consider upgrading via the 'c:\users\nl\testweb\flask_env\scripts\python.exe -m p
ip install --upgrade pip' command.

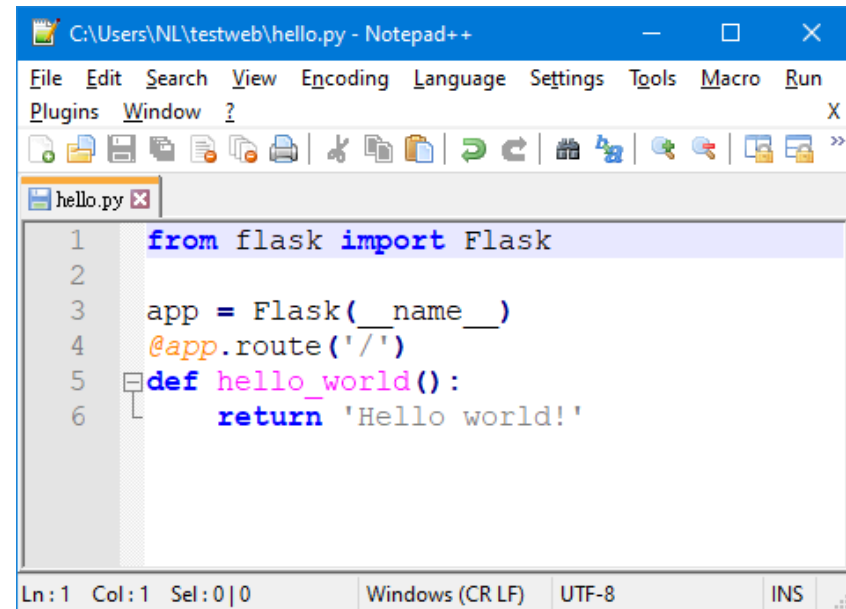
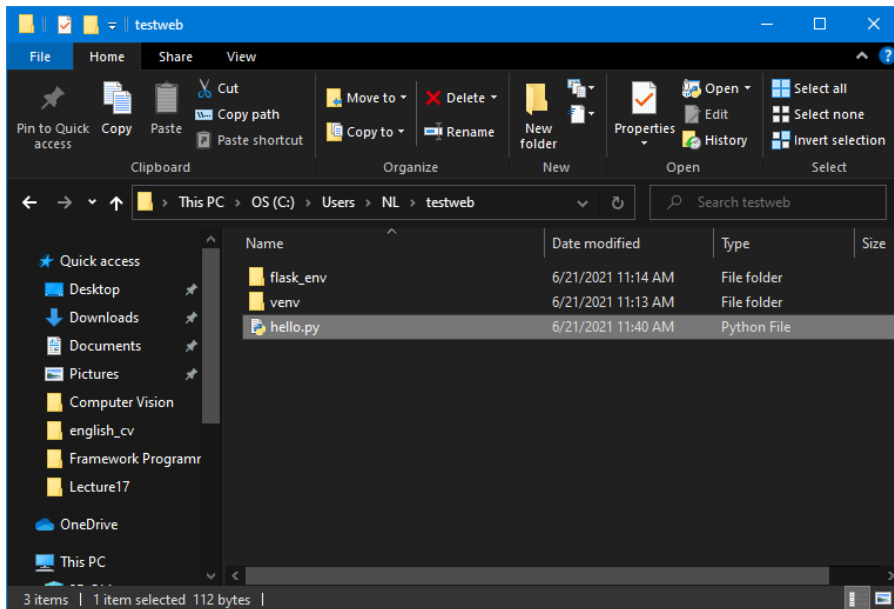
(flask_env) (semmy_env) C:\Users\NL\testweb>_
```



# Step 5: Test the Development Environment

- 1) Create a simple **Flask application to test** the newly created development environment.
- 2) Make a file in the Flask project folder called **hello.py**.
- 3) Edit the file using a text editor and add the following code to make an application that prints “*Hello world!*”.
- 4) **Note:** Pick any name for the project except flask.py. The Flask library is in a flask.py file.

**Dir path:** C:\Users\NL\testweb



# Set FLASK\_APP *environment variable*

**Open new console (CMD)** to navigate to the project folder using the cd command.

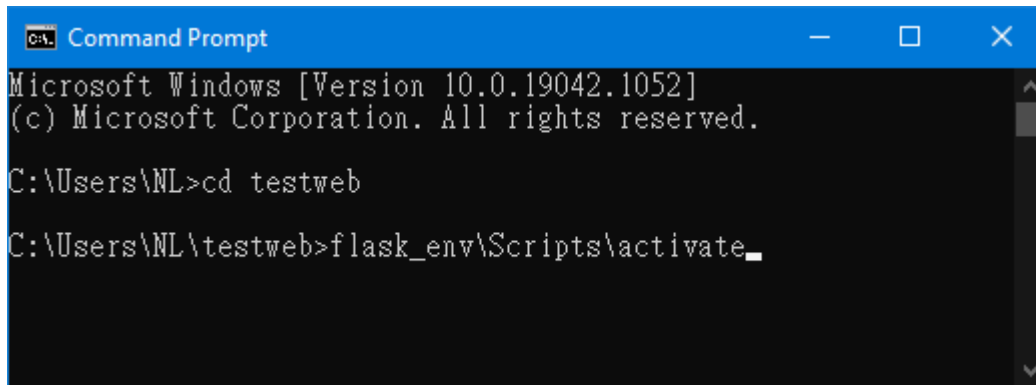
Set the FLASK\_APP environment variable.

## 1. For Linux and Mac:

```
$ export FLASK_APP=hello.py
```

## 2. For Windows:

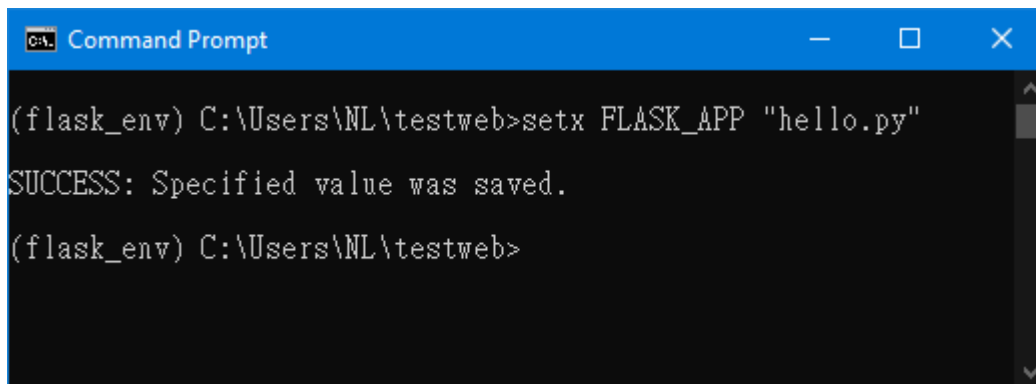
```
$ setx FLASK_APP "hello.py"
```



```
Command Prompt
Microsoft Windows [Version 10.0.19042.1052]
(c) Microsoft Corporation. All rights reserved.

C:\Users\NL>cd testweb

C:\Users\NL\testweb>flask_env\Scripts\activate_
```



```
Command Prompt

(flask_env) C:\Users\NL\testweb>setx FLASK_APP "hello.py"

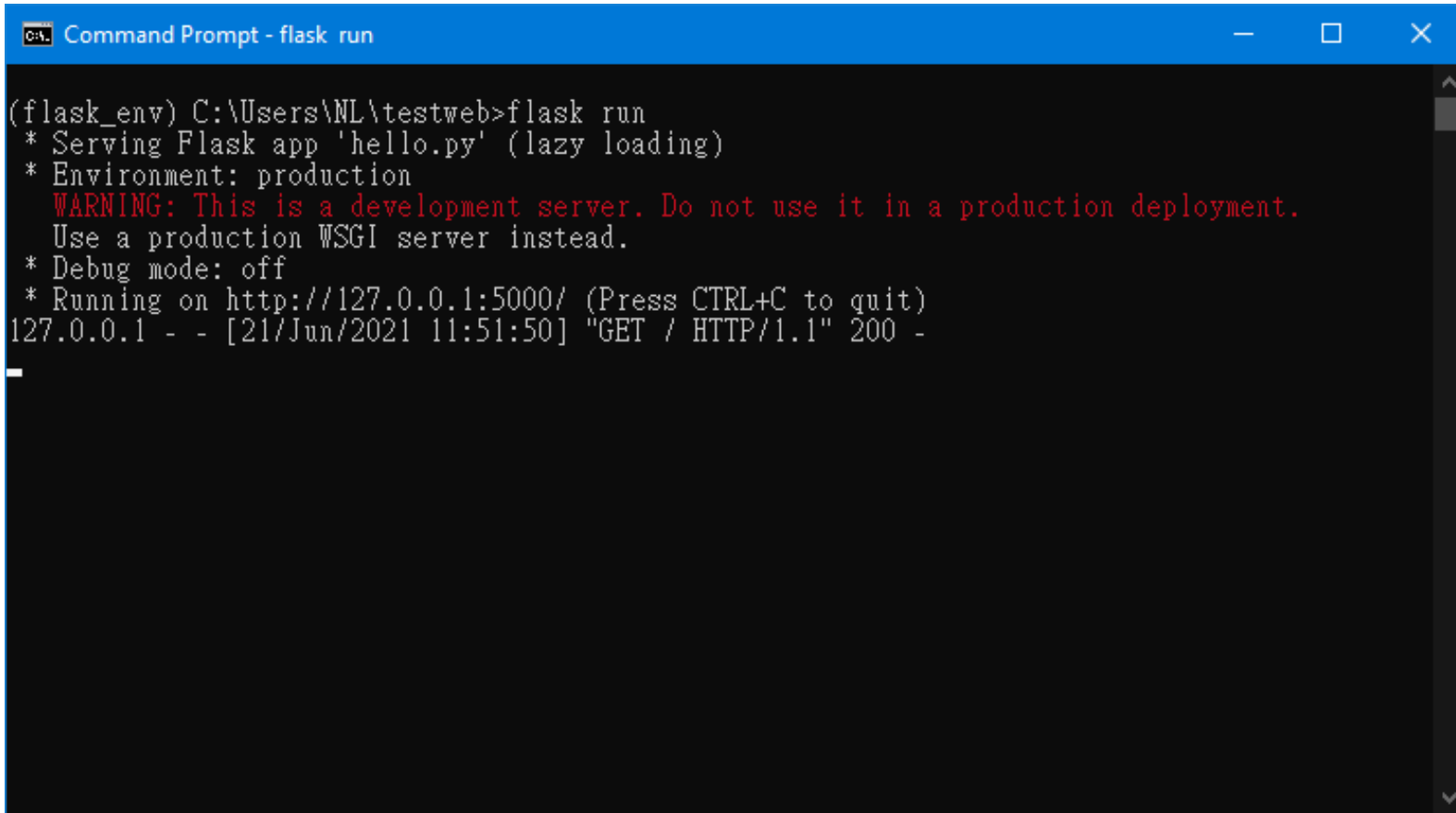
SUCCESS: Specified value was saved.

(flask_env) C:\Users\NL\testweb>
```

# Run the Flask Application

Run the Flask application with:

\$ *flask run*



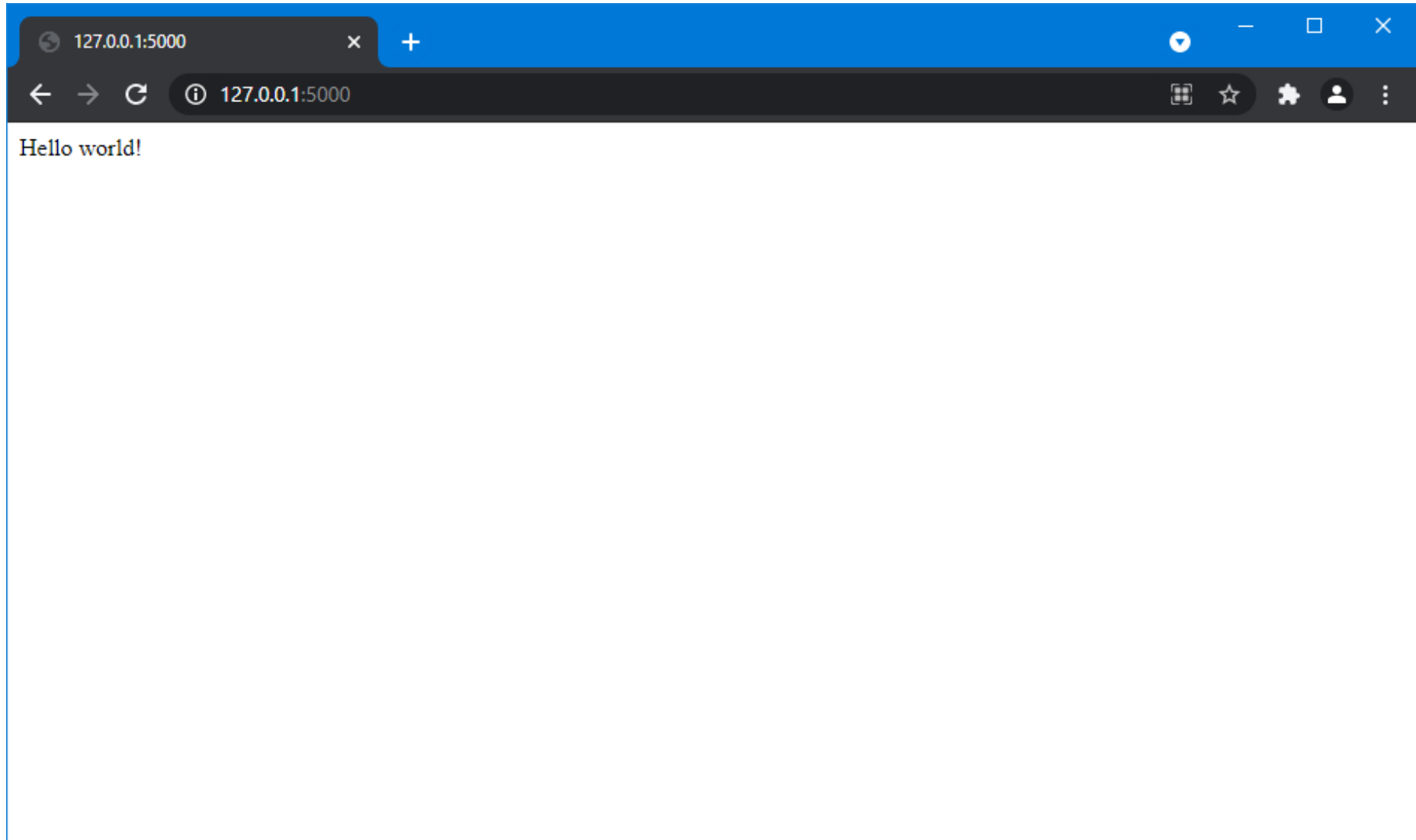
```
Command Prompt - flask run

(flask_env) C:\Users\NL\testweb>flask run
* Serving Flask app 'hello.py' (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
127.0.0.1 - - [21/Jun/2021 11:51:50] "GET / HTTP/1.1" 200 -
```

Copy and paste this URL in your browser: *http://127.0.0.1:5000/*

# Run the Flask Application

Check your browser:



# After Run Flask Application

**Note:** Windows users must restart/close the console (CMD) to set the environment variable.

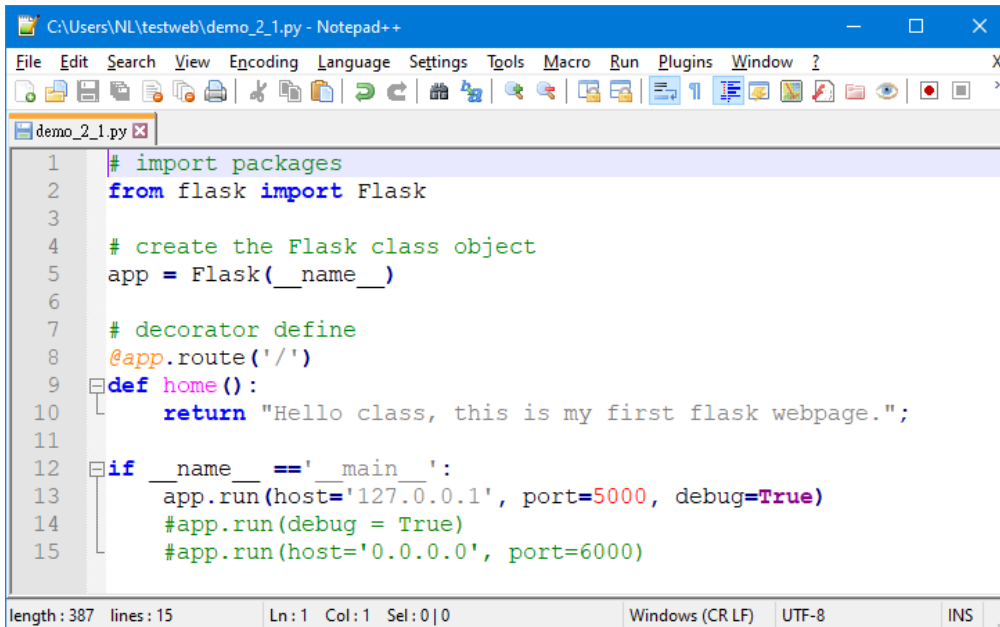


(#2) exercise for students  
**[Web Framework]**

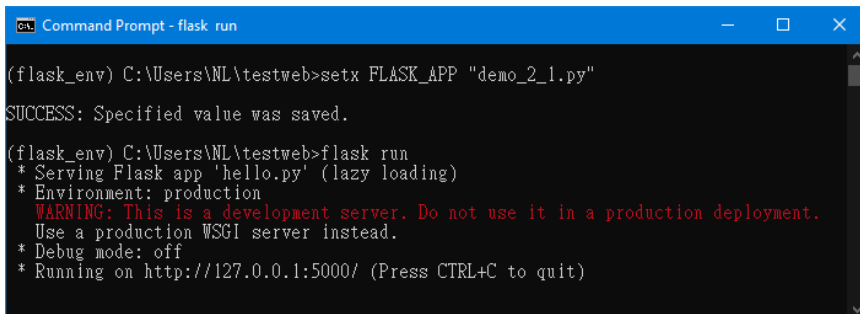
# Exercise #2.1

File : *demo\_2\_1.py*

Desc : Display/create/build first python website using the Flask framework.



```
1 # import packages
2 from flask import Flask
3
4 # create the Flask class object
5 app = Flask(__name__)
6
7 # decorator define
8 @app.route('/')
9 def home():
10     return "Hello class, this is my first flask webpage.";
11
12 if __name__ == '__main__':
13     app.run(host='127.0.0.1', port=5000, debug=True)
14     #app.run(debug = True)
15     #app.run(host='0.0.0.0', port=6000)
```

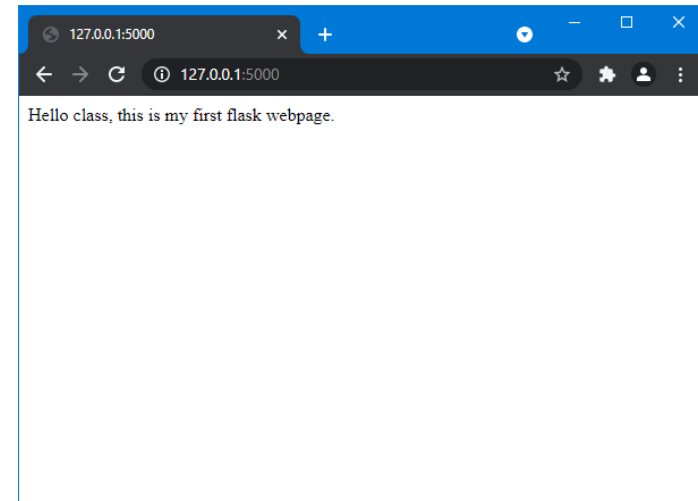


```
Command Prompt - flask run

(flask_env) C:\Users\NL\testweb>setx FLASK_APP "demo_2_1.py"
SUCCESS: Specified value was saved.

(flask_env) C:\Users\NL\testweb>flask run
* Serving Flask app 'hello.py' (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
```

**Output Program:**



# Exercise #2.2

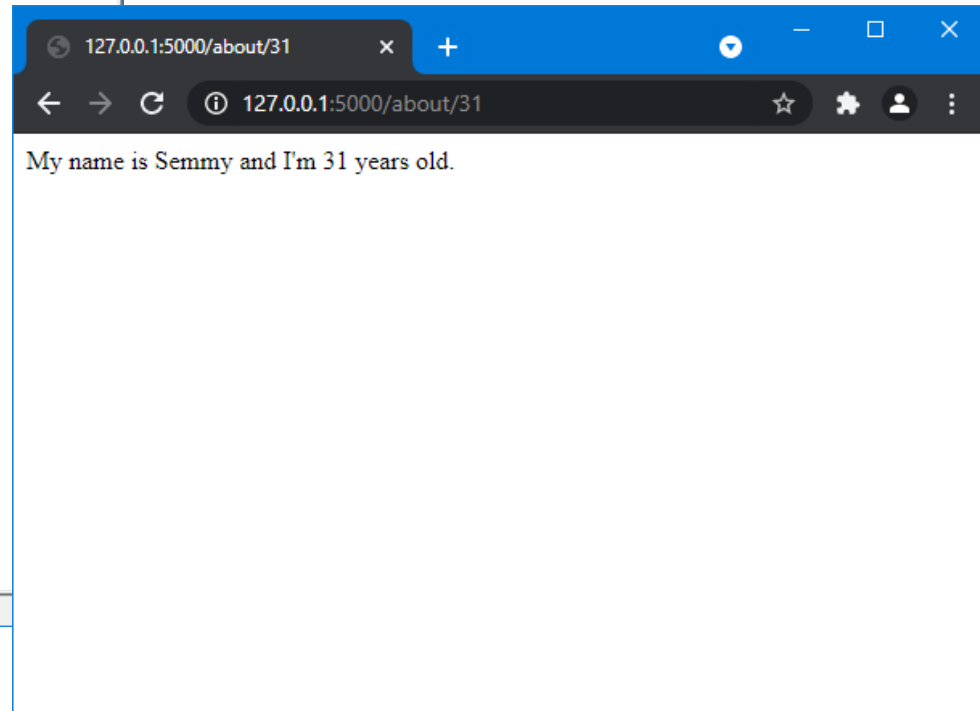
File : *demo\_2\_3.py*

Desc : Flask App routing

```
C:\Users\NL\testweb\demo_2_2.py - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
demo_2_2.py
1  # import packages
2  from flask import Flask
3
4  # create the Flask class object
5  app = Flask(__name__)
6
7  # flask router default
8  @app.route("/home/<name>")
9  def home(name):
10     if name is not None:
11         return "My name is "+name+ ".";
12     else:
13         return "My name is Semmy.";
14
15  # flask router about
16  @app.route("/about/<int:age>")
17  def about(age):
18     return "My name is Semmy and I'm %d years old."%age;
19
20  # flask router project
21  @app.route("/project")
22  def project():
23     return "This is project :)"
24
25  if __name__ == "__main__":
26     app.run(debug=True)

length: 600 lines: 26 Ln: 1 Col: 1 Sel: 0|0 Windows (CR LF) UTF-8
```

**Output Program:**





# Exercise #2.3

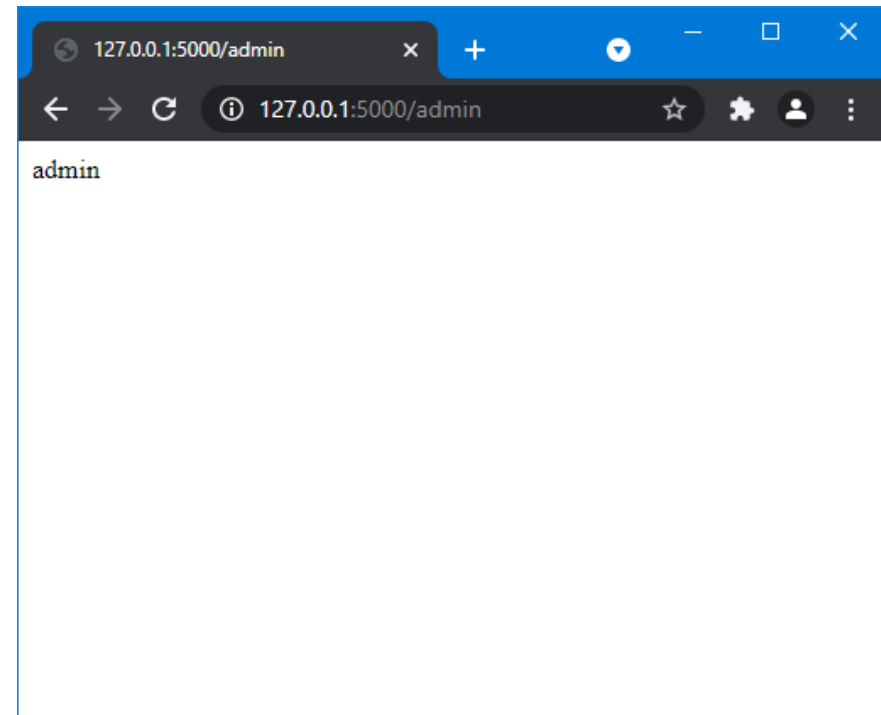
File : *demo\_2\_3.py*

Desc : Flask App routing

```
C:\Users\NL\testweb\demo_2_3.py - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
demo_2_3.py
1  # import packages
2  from flask import Flask
3
4  # create the Flask class object
5  app = Flask(__name__)
6
7  # flask router admin
8  @app.route('/admin')
9  def admin():
10     return 'admin'
11
12  @app.route('/librarian')
13  def librarian():
14     return 'librarian'
15
16  @app.route('/student')
17  def student():
18     return 'student'
19
20  @app.route('/user/<name>')
21  def user(name):
22     if name == 'admin':
23         return redirect(url_for('admin'))
24     if name == 'librarian':
25         return redirect(url_for('librarian'))
26     if name == 'student':
27         return redirect(url_for('student'))
28
29  # Main
30  if __name__ == "__main__":
31     app.run(debug=True)

length: 690 lines: 31 Ln: 1 Col: 1 Sel: 0|0 Windows (CR LF) UTF-8 INS
```

**Output Program:**



# Exercise #2.4

File : *demo\_2\_4.py*

Desc : Python3, Flask and OpenCV

Create an example for openCV object detection with python flask application.

# Reference (GUI Frameworks)

## **How To Install Tkinter In Windows**

<https://www.activestate.com/resources/quick-reads/how-to-install-tkinter-in-windows/>

## **How to install Tkinter in Python?**

<https://www.tutorialspoint.com/how-to-install-tkinter-in-python>

## **Python GUI Programming With Tkinter**

<https://realpython.com/python-gui-tkinter/>

## **Python - GUI Programming (Tkinter)**

[https://www.tutorialspoint.com/python/python\\_gui\\_programming.htm](https://www.tutorialspoint.com/python/python_gui_programming.htm)

## **Introduction to GUI With Tkinter in Python**

<https://www.datacamp.com/community/tutorials/gui-tkinter-python>

## **Python Tkinter Tutorial**

<https://www.geeksforgeeks.org/python-tkinter-tutorial/>

## **Introduction to GUI programming with tkinter**

[https://python-textbok.readthedocs.io/en/1.0/Introduction\\_to\\_GUI\\_Programming.html](https://python-textbok.readthedocs.io/en/1.0/Introduction_to_GUI_Programming.html)

## **Create UI in Python-Tkinter**

<https://www.tutorialsteacher.com/python/create-gui-using-tkinter-python>

## **Python GUI examples (Tkinter Tutorial)**

<https://likegeeks.com/python-gui-examples-tkinter-tutorial/>

## **Python GUI – tkinter**

<https://www.geeksforgeeks.org/python-gui-tkinter/>

# Reference (Web Frameworks)

## **Python Flask Tutorial – Introduction To Flask For Beginners**

<https://www.softwaretestinghelp.com/python-flask-tutorial/>

## **Flask Mega-Tutorial**

<https://blog.miguelgrinberg.com/post/the-flask-mega-tutorial-part-i-hello-world>

## **Why is Flask a good web framework choice?**

<https://www.fullstackpython.com/flask.html>

## **Python Flask Windows Development Environment Setup**

<https://timmyreilly.azurewebsites.net/python-flask-windows-development-environment-setup/>

## **Install Flask and create your first web application**

<https://dev.to/sahilrajput/install-flask-and-create-your-first-web-application-2dba>

## **How to install FLASK in Windows 10**

<https://www.osradar.com/how-to-install-flask-in-windows-10/>

## **How To Install Flask**

<https://phoenixnap.com/kb/install-flask>

## **How To Make a Web Application Using Flask in Python**

<https://www.digitalocean.com/community/tutorials/how-to-make-a-web-application-using-flask-in-python-3>

## **Flask Tutorials**

<https://realpython.com/tutorials/flask/>

## **What is Flask?**

[https://www.tutorialspoint.com/flask/flask\\_overview.htm](https://www.tutorialspoint.com/flask/flask_overview.htm)

## **Python Flask Tutorial**

<https://www.javatpoint.com/flask-tutorial>

## **How to build a web application using Flask**

<https://www.freecodecamp.org/news/how-to-build-a-web-application-using-flask-and-deploy-it-to-the-cloud-3551c985e492/>

# END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

